ROOTS OF DEWANGAN

Complete Software Requirements Specification (SRS)

Version 2.0 - Comprehensive Edition

Document Control

Attribute	Value
Project Name	Roots of Dewangan Heritage Operating System
Version	2.0 Complete
Date	February 5, 2026
Status	Final Specification
Scope	Multi-generational family documentation system
Lifespan	100+ years
Target Scale	1,000+ members, 10,000+ media items

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY

1.1 Project Overview

Roots of Dewangan is a comprehensive, AI-powered, multi-generational family heritage documentation system designed to preserve, organize, and analyze family history for 100+ years.

Key Characteristics:

- **Zero-cost infrastructure** using free tiers of production-grade services
- **AI-native architecture** with built-in intelligence from day 1
- **Mobile-first design** optimized for smartphone usage
- **Unlimited scalability** from 50 to 100,000+ family members
- **Future-proof** with open standards and data portability

1.2 Problem Statement

Traditional family history preservation faces critical challenges:

1. **Data Loss**: Photos deteriorate, stories forgotten when elders pass
2. **Fragmentation**: Information scattered across albums, memories, documents
3. **Inaccessibility**: Data locked in one person's possession
4. **Non-collaboration**: Family members can't contribute easily
5. **No Analysis**: No insights from patterns (occupation trends, migrations)
6. **Technology Gap**: Non-technical family members excluded

1.3 Solution

A cloud-based, collaborative platform that:

✅ Captures family data through intuitive forms ✅ Preserves media with automatic compression ✅ Links relationships intelligently ✅ Enables AI-powered insights ✅ Provides mobile-first access ✅ Scales infinitely at zero cost ✅ Exports data in open formats

1.4 Success Metrics

Year 1 Targets:

- 100+ family members documented

- 500+ media items uploaded
- 50+ stories preserved
- 10+ active contributors
- 80%+ data completeness

Year 5 Targets:

- 300+ family members
- 3,000+ media items
- 200+ stories
- 1 AI-generated family book
- Migration map with 5+ locations

1.5 Key Stakeholders

Role	Responsibility	Access Level
Project Owner	Overall vision, funding decisions	Super Admin
Primary Admin	Daily operations, data entry	Admin
Secondary Admin	Backup management, submissions review	Admin
Family Contributors	Submit data via forms	Contributor
Family Viewers	Browse family tree, read stories	Viewer
Future Generations	Inherit and extend system	TBD

2. PROJECT VISION & GOALS

2.1 Vision Statement

"To create an eternal digital sanctuary where every family member—past, present, and future—is remembered, connected, and celebrated through technology that outlasts us all."

2.2 Core Principles

Principle 1: Longevity First

- Every design decision prioritizes 100+ year lifespan
- Open formats, exportable data, no vendor lock-in
- Graceful degradation if services shut down

Principle 2: Inclusive Access

- Non-technical elders can contribute easily
- Mobile-optimized for smartphone-only users
- Multiple languages supported

Principle 3: Data Integrity

- Preserve truth, acknowledge uncertainty
- Audit trails for all changes
- Dispute resolution mechanisms

Principle 4: Privacy Respecting

- Consent-based sharing
- Sensitive data encrypted
- Living members control their information

Principle 5: AI-Augmented, Human-Verified

- AI assists but doesn't replace human judgment
- All AI outputs clearly labeled
- Human approval required for critical data

2.3 Objectives

Primary Objectives:

1. **Preservation**: Digitize and protect 4 generations of family history
2. **Collaboration**: Enable 20+ family members to contribute
3. **Accessibility**: Provide 24/7 access from any device
4. **Intelligence**: Generate insights humans couldn't see manually
5. **Legacy**: Create artifact for descendants 100 years from now

Secondary Objectives:

1. Pattern Recognition: Identify occupation, education, migration trends
2. Cultural Documentation: Preserve traditions, recipes, customs
3. Community Building: Strengthen family bonds across geography
4. Education: Teach younger generations about heritage
5. Celebration: Honor achievements and overcome hardships

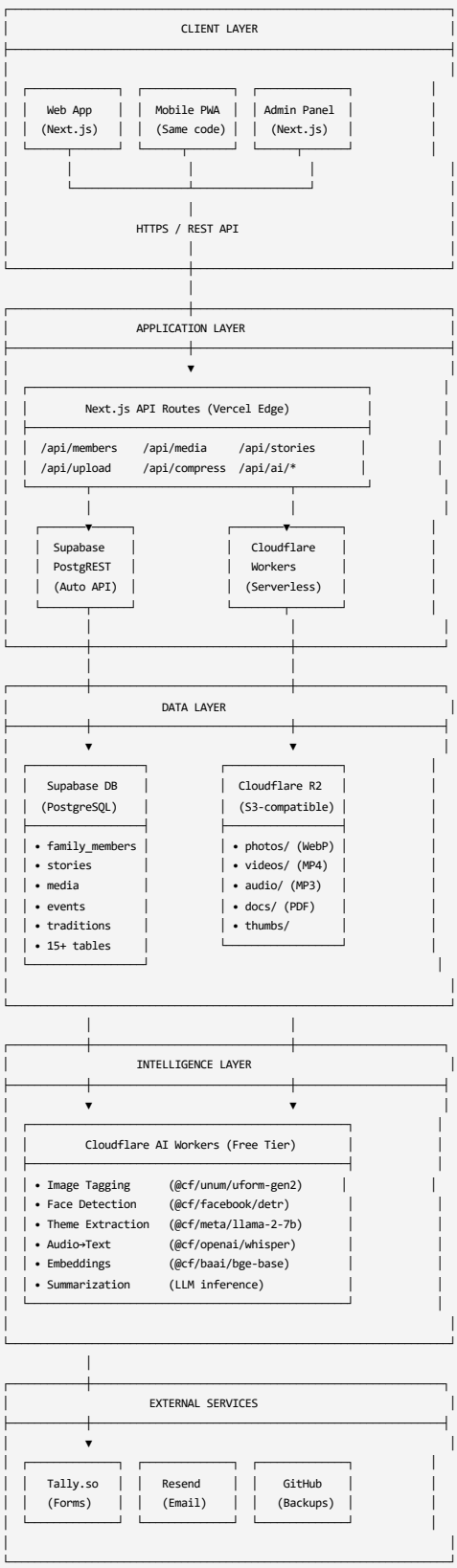
2.4 Non-Goals

What this project is **NOT**:

❌ Social network competitor (not Facebook for family) ❌ Genealogy research tool (not for finding distant relatives) ❌ Commercial product (private family use only) ❌ Real-time communication platform (not WhatsApp replacement) ❌ Photo editing suite (basic compression only)

3. SYSTEM ARCHITECTURE

3.1 High-Level Architecture Diagram



3.2 Architecture Patterns

Pattern 1: JAMstack Architecture

- **J**avaScript (React/Next.js frontend)

- **A**PIs (Supabase REST, Cloudflare Workers)
- **M**arkup (Static site generation where possible)

Benefits:

- Fast: Pre-rendered pages, CDN distribution
- Secure: No server to hack, APIs isolated
- Scalable: Static assets scale infinitely
- Cost: Free hosting on Vercel

Pattern 2: Serverless-First

- No traditional servers
- Functions execute on-demand
- Auto-scaling
- Pay-per-use (mostly free tier)

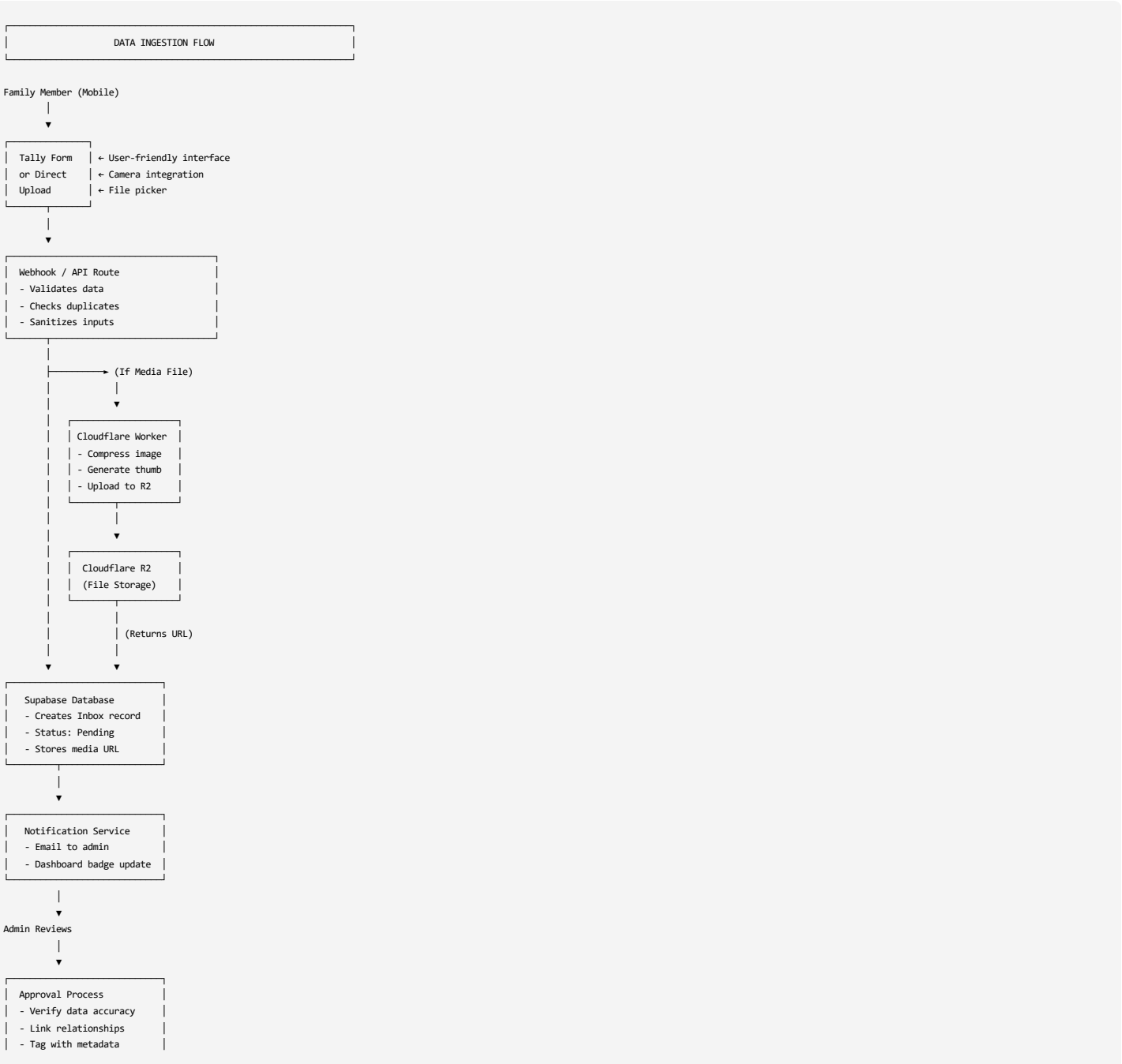
Pattern 3: Edge Computing

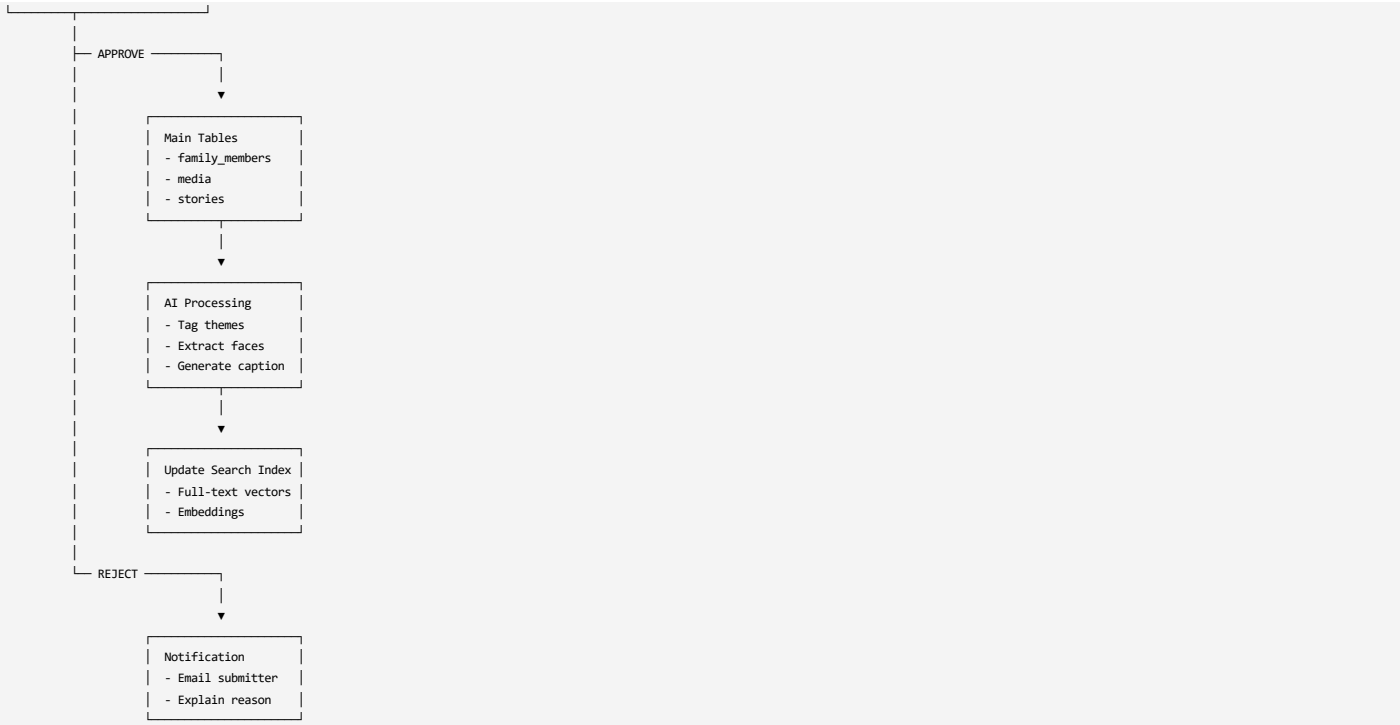
- Code runs close to users (Cloudflare global network)
- Faster response times
- Image optimization at edge

Pattern 4: Event-Driven

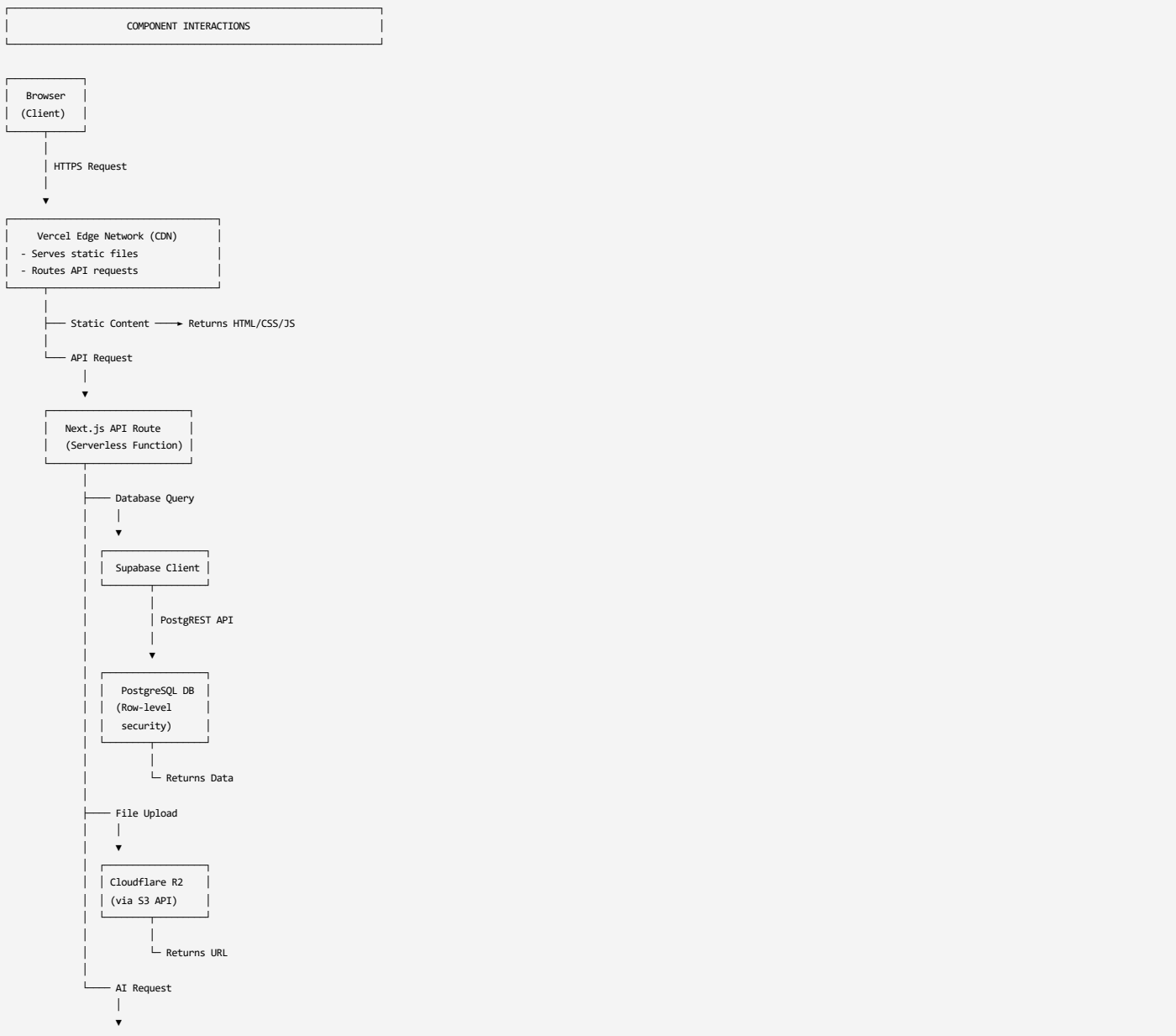
- Form submission → triggers webhook → creates inbox record
- Media upload → triggers compression → stores in R2
- AI request → queues job → processes asynchronously

3.3 Data Flow Architecture





3.4 Component Interaction Diagram



```
                ┌──────────────┐
                │ Cloudflare AI │
                │ Workers       │
                └──────┬────────┘
                       │
                       └─ Returns Inference

All components return to Next.js API Route
       │
       │ Aggregates responses
       │
       ▼
Returns JSON to Client
```

3.5 Deployment Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                    DEPLOYMENT TOPOLOGY                       │
└──────────────────────────────────────────────────────────────┘

Developer Laptop
       │
       │ git push
       │
       ▼
┌──────────────┐
│ GitHub Repo  │ ← Source of truth
│ (Private)    │ ← Version control
└──────┬───────┘
       │
       │ Webhook triggers
       │
       ▼
┌──────────────────────────────────┐
│      Vercel CI/CD Pipeline       │
│ 1. Pull code from GitHub         │
│ 2. Install dependencies (npm)    │
│ 3. Run build (next build)        │
│ 4. Run tests (optional)          │
│ 5. Deploy to Edge Network        │
└──────┬───────────────────────────┘
       │
       │ Auto-deploys to:
       │
       ▼
┌──────────────────────────────────────────────┐
│        Vercel Global Edge Network            │
│  ┌──────────┐ ┌──────────┐ ┌──────────┐      │
│  │ Region   │ │ Region   │ │ Region   │ ...  │
│  │   US     │ │   EU     │ │ APAC     │      │
│  └──────────┘ └──────────┘ └──────────┘      │
│  (Closest to user = fastest response)        │
└──────────────────────────────────────────────┘

External Services (Already Deployed):

┌──────────────┐ ┌──────────────┐ ┌──────────────┐
│ Supabase     │ │ Cloudflare R2 │ │ Cloudflare AI │
│ (us-east-1)  │ │ (Global)      │ │ (Global)      │
│ - PostgreSQL │ │ - File storage │ │ - AI inference │
└──────────────┘ └──────────────┘ └──────────────┘

Production URL: https://roots-of-dewangan.vercel.app
Preview URL: https://roots-of-dewangan-git-[branch]-[user].vercel.app
```

4. TECHNOLOGY STACK

4.1 Complete Technology Matrix

Layer	Technology	Version	Purpose	Cost	Alternatives
Frontend	Next.js	14.x	React framework, SSR/SSG	Free	Remix, Astro
	React	18.x	UI library	Free	Vue, Svelte
	TypeScript	5.x	Type safety	Free	JavaScript
	Tailwind CSS	3.x	Styling	Free	CSS-in-JS
	shadcn/ui	Latest	Component library	Free	Material-UI
	React Flow	11.x	Family tree visualization	Free	D3.js
	Recharts	2.x	Analytics charts	Free	Chart.js
Backend	Supabase	Cloud	Database + Auth + Storage	Free tier	Firebase
	PostgreSQL	15.x	Relational database	Free	MySQL
	PostgREST	Auto	Auto REST API	Free	Custom API
Storage	Cloudflare R2	-	Object storage (S3-compat)	Free 10GB	AWS S3
Serverless	Vercel Functions	-	API routes, serverless	Free	AWS Lambda
	Cloudflare Workers	-	Edge compute, AI	Free 10K/day	Deno Deploy
AI/ML	Cloudflare AI	-	Image/text models	Free 10K/day	OpenAI API
	Whisper	-	Audio transcription	Free	Google STT
	LLaMA 2	7B	Theme extraction	Free	GPT-3.5

Layer	Technology	Version	Purpose	Cost	Alternatives
Forms	Tally.so	-	User submissions	Free	Typeform
Email	Resend	-	Transactional emails	Free 3K/mo	SendGrid
Auth	Supabase Auth	-	User authentication	Free	Auth0
Deployment	Vercel	-	Hosting, CDN, CI/CD	Free	Netlify
Version Control	GitHub	-	Code repository	Free	GitLab
Monitoring	Vercel Analytics	-	Performance monitoring	Free	Google Analytics
Backups	GitHub Actions	-	Automated backups	Free	Cron jobs

4.2 Technology Decision Rationale

Why Next.js?

✅ Full-stack framework (frontend + API routes) ✅ Excellent SEO (server-side rendering) ✅ Image optimization built-in ✅ File-based routing (simple) ✅ Huge ecosystem, great docs ✅ Vercel integration (deploy in seconds)

Alternatives Considered:

- ❌ Create React App: No SSR, no API routes
- ❌ Remix: Newer, smaller ecosystem
- ❌ Astro: Content-focused, less dynamic

Why Supabase?

✅ Open-source (not locked in) ✅ PostgreSQL (most powerful open DB) ✅ Real-time subscriptions (live updates) ✅ Row-level security (built-in auth) ✅ Auto-generated REST API ✅ 500MB free forever

Alternatives Considered:

- ❌ Firebase: NoSQL (harder relationships), vendor lock-in
- ❌ PlanetScale: MySQL (less features than Postgres)
- ❌ Self-hosted Postgres: Maintenance burden

Why Cloudflare R2?

✅ Zero egress fees (unlike AWS S3) ✅ S3-compatible API (easy migration) ✅ 10GB free storage ✅ Global CDN included ✅ Fast uploads/downloads

Alternatives Considered:

- ❌ AWS S3: Expensive egress ($0.09/GB)
- ❌ Google Cloud Storage: More complex
- ❌ Supabase Storage: Only 1GB free

Why Cloudflare AI Workers?

✅ 10,000 free requests/day ✅ No API keys needed ✅ Models run at edge (fast) ✅ Whisper, LLaMA, Vision models included ✅ No rate limits on free tier (just daily cap)

Alternatives Considered:

- ❌ OpenAI API: $0.002/1K tokens (costs add up)
- ❌ Google Cloud AI: Complex setup
- ❌ Hugging Face: Need to host models

4.3 Technology Integration Map

```
    │
    └── API Routes (/app/api/*)
        │
        ├─ /api/members
        │  │
        │  └─→ Supabase Client
        │      - CRUD operations
        │      - Uses PostgREST API
        │      - Returns JSON
        │
        ├─ /api/upload
        │  │
        │  └─→ AWS SDK (S3-compatible)
        │      - Connects to R2
        │      - PUT object
        │      - Returns public URL
        │
        ├─ /api/compress
        │  │
        │  └─→ Sharp (image library)
        │      - Resize image
        │      - Convert to WebP
        │      - Reduces file size
        │
        └─ /api/ai/tag-image
           │
           └─→ Cloudflare AI Workers
               - Fetch API to CF endpoint
               - Send image buffer
               - Receive tags/caption

External Form (Tally)
    │
    │ On submit
    │
    └─→ Webhook to /api/webhook/submission
        │
        └─→ Validates & inserts to Supabase inbox table
```

4.4 Environment Variables Configuration

```
# File: .env.local (Never commit to Git!)

# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGc...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGc...  # Server-side only

# Cloudflare R2
R2_ACCOUNT_ID=abc123...
R2_ACCESS_KEY_ID=def456...
R2_SECRET_ACCESS_KEY=ghi789...
R2_BUCKET_NAME=roots-of-dewangan
NEXT_PUBLIC_R2_PUBLIC_URL=https://pub-xxx.r2.dev

# Cloudflare AI
CLOUDFLARE_ACCOUNT_ID=abc123...
CLOUDFLARE_AI_TOKEN=xyz789...

# Resend (Email)
RESEND_API_KEY=re_123...
RESEND_FROM_EMAIL=noreply@roots-dewangan.com

# App Config
NEXT_PUBLIC_APP_URL=https://roots-of-dewangan.vercel.app
ADMIN_EMAILS=admin@example.com,backup@example.com

# Webhook Secrets (for Tally)
TALLY_WEBHOOK_SECRET=secret123...

# GitHub (for backups)
GITHUB_TOKEN=ghp_123...
GITHUB_BACKUP_REPO=username/roots-backups
```

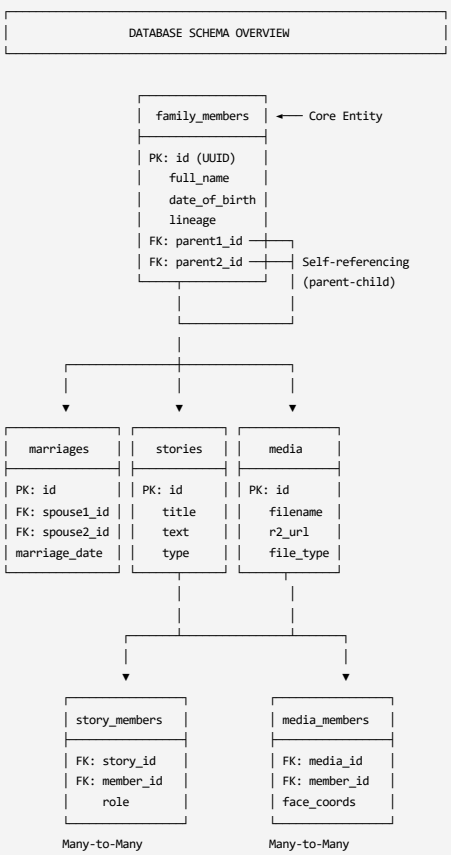
4.5 Package Dependencies

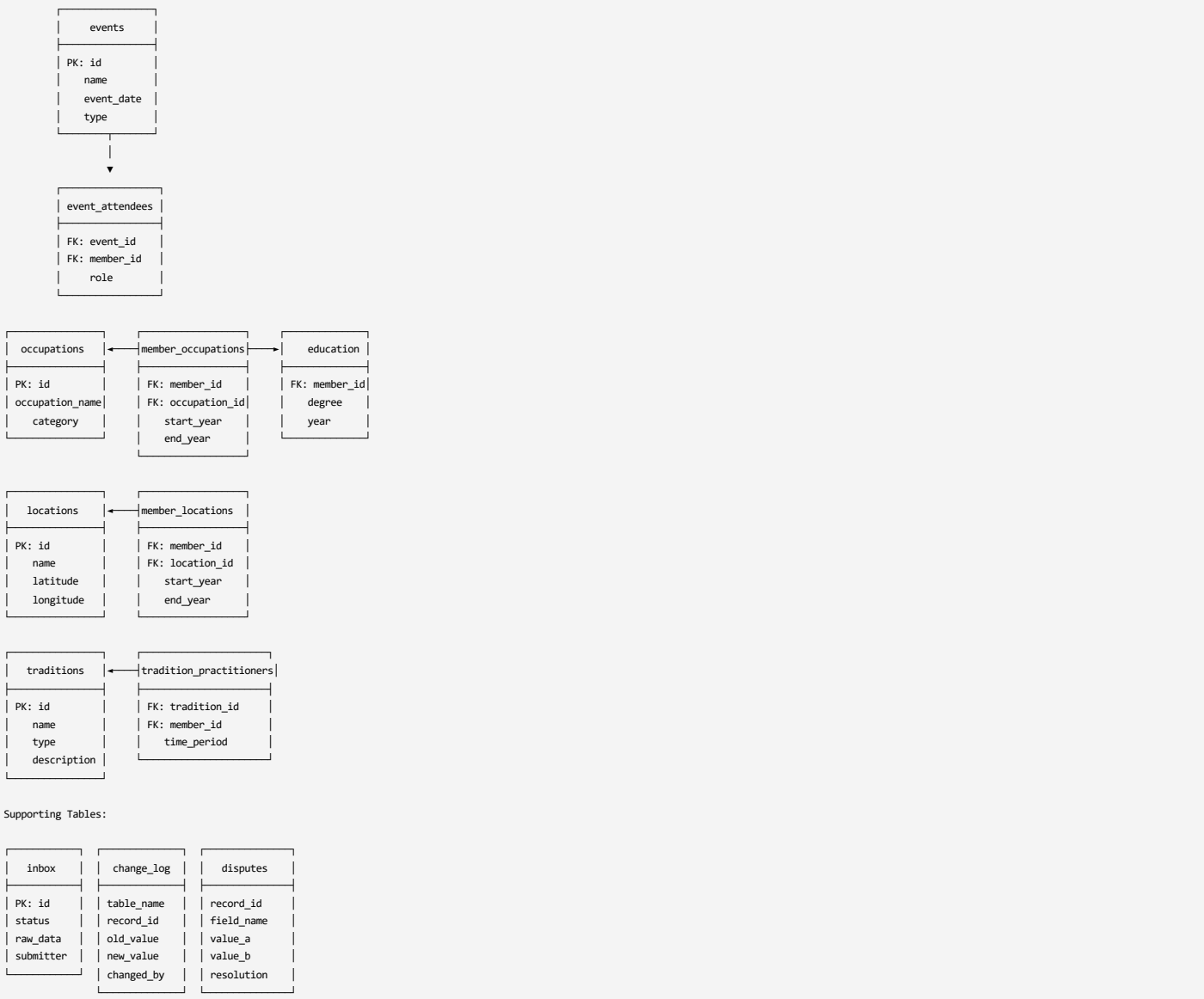
```
// File: package.json

{
  "name": "roots-of-dewangan",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "type-check": "tsc --noEmit"
  },
  "dependencies": {
    "next": "^14.1.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",

    // Supabase
```

```
    "@supabase/supabase-js": "^2.39.0",
    "@supabase/ssr": "^0.0.10",

    // UI Components
    "@radix-ui/react-dialog": "^1.0.5",
    "@radix-ui/react-dropdown-menu": "^2.0.6",
    "@radix-ui/react-select": "^2.0.0",
    "lucide-react": "^0.316.0",

    // Styling
    "tailwindcss": "^3.4.1",
    "class-variance-authority": "^0.7.0",
    "clsx": "^2.1.0",
    "tailwind-merge": "^2.2.0",

    // Forms
    "react-hook-form": "^7.49.3",
    "zod": "^3.22.4",
    "@hookform/resolvers": "^3.3.4",

    // Data Visualization
    "react-flow-renderer": "^10.3.17",
    "recharts": "^2.10.4",
    "leaflet": "^1.9.4",
    "react-leaflet": "^4.2.1",

    // Image Processing
    "sharp": "^0.33.2",

    // File Upload
    "@aws-sdk/client-s3": "^3.490.0",
    "@aws-sdk/s3-request-presigner": "^3.490.0",

    // Utilities
    "date-fns": "^3.3.1",
    "uuid": "^9.0.1",
    "fuse.js": "^7.0.0"  // Fuzzy search
  },
  "devDependencies": {
    "typescript": "^5.3.3",
    "@types/node": "^20.11.5",
    "@types/react": "^18.2.48",
    "@types/react-dom": "^18.2.18",
    "autoprefixer": "^10.4.17",
    "postcss": "^8.4.33",
    "eslint": "^8.56.0",
    "eslint-config-next": "^14.1.0"
  }
}
```

5. DATABASE DESIGN

5.1 Entity-Relationship Diagram (ERD)





5.2 Complete Table Schemas

Table: family_members

```
CREATE TABLE family_members (
  -- Primary Key
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- Basic Info
  full_name TEXT NOT NULL,
  nickname TEXT,
  gender TEXT CHECK (gender IN ('Male', 'Female', 'Other')),
  date_of_birth DATE,
  date_of_death DATE,

  -- Location
  birth_place TEXT,
  current_location TEXT,

  -- Classification
  lineage TEXT CHECK (lineage IN ('Father', 'Mother', 'Both')) NOT NULL,
  status TEXT CHECK (status IN ('Living', 'Deceased')) DEFAULT 'Living',
  generation INTEGER,

  -- Relationships (self-referencing foreign keys)
  parent1_id UUID REFERENCES family_members(id) ON DELETE SET NULL,
  parent2_id UUID REFERENCES family_members(id) ON DELETE SET NULL,

  -- Rich Content
  bio_summary TEXT,
  profile_photo_url TEXT,

  -- Sensitive Data (encrypted)
  contact_info JSONB, -- {email: "...", phone: "..."}

  -- Metadata
  added_by TEXT,
  added_date TIMESTAMPTZ DEFAULT NOW(),
  last_modified TIMESTAMPTZ DEFAULT NOW(),
```

```
  -- Full-Text Search
  search_vector tsvector GENERATED ALWAYS AS (
    to_tsvector('english',
      coalesce(full_name, '') || ' ' ||
      coalesce(nickname, '') || ' ' ||
      coalesce(bio_summary, '')
    )
  ) STORED,

  -- Constraints
  CONSTRAINT valid_dates CHECK (
    date_of_birth IS NULL OR
    date_of_death IS NULL OR
    date_of_death > date_of_birth
  ),
  CONSTRAINT no_self_parent CHECK (
    id != parent1_id AND id != parent2_id
  )
);

-- Indexes
CREATE INDEX idx_family_members_name ON family_members(full_name);
CREATE INDEX idx_family_members_lineage ON family_members(lineage);
CREATE INDEX idx_family_members_generation ON family_members(generation);
CREATE INDEX idx_family_members_status ON family_members(status);
CREATE INDEX idx_family_members_parent1 ON family_members(parent1_id);
CREATE INDEX idx_family_members_parent2 ON family_members(parent2_id);
CREATE INDEX idx_family_members_search ON family_members USING GIN(search_vector);
CREATE INDEX idx_family_members_dob ON family_members(date_of_birth);

-- Computed Fields (Virtual Columns via Functions)
CREATE OR REPLACE FUNCTION get_age(dob DATE, dod DATE)
RETURNS INTEGER AS $$
BEGIN
  IF dod IS NOT NULL THEN
    RETURN EXTRACT(YEAR FROM AGE(dod, dob));
  ELSIF dob IS NOT NULL THEN
    RETURN EXTRACT(YEAR FROM AGE(CURRENT_DATE, dob));
  END IF;
  RETURN NULL;
END;
$$ LANGUAGE plpgsql IMMUTABLE;

-- Trigger for last_modified
CREATE OR REPLACE FUNCTION update_modified_column()
RETURNS TRIGGER AS $$
BEGIN
  NEW.last_modified = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_family_members_modified
BEFORE UPDATE ON family_members
FOR EACH ROW EXECUTE FUNCTION update_modified_column();
```

Table: marriages

```
CREATE TABLE marriages (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- Spouses (both required)
  spouse1_id UUID NOT NULL REFERENCES family_members(id) ON DELETE CASCADE,
  spouse2_id UUID NOT NULL REFERENCES family_members(id) ON DELETE CASCADE,

  -- Details
  marriage_date DATE,
  divorce_date DATE,
  marriage_location TEXT,
  status TEXT CHECK (status IN ('Married', 'Divorced', 'Widowed')) DEFAULT 'Married',

  -- Notes
  notes TEXT,

  -- Metadata
  added_by TEXT,
  added_date TIMESTAMPTZ DEFAULT NOW(),

  -- Constraints
  CONSTRAINT different_spouses CHECK (spouse1_id != spouse2_id),
  CONSTRAINT valid_marriage_dates CHECK (
    divorce_date IS NULL OR
    marriage_date IS NULL OR
    divorce_date > marriage_date
  ),
  CONSTRAINT unique_marriage UNIQUE (spouse1_id, spouse2_id, marriage_date)
);

CREATE INDEX idx_marriages_spouse1 ON marriages(spouse1_id);
CREATE INDEX idx_marriages_spouse2 ON marriages(spouse2_id);
CREATE INDEX idx_marriages_status ON marriages(status);
```

Table: media

```
CREATE TABLE media (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
```

```
  -- File Info
  filename TEXT NOT NULL,
  file_type TEXT CHECK (file_type IN ('Photo', 'Video', 'Audio', 'Document')) NOT NULL,

  -- Storage
  r2_key TEXT NOT NULL UNIQUE, -- Path in R2: photos/2024/wedding.webp
  r2_url TEXT NOT NULL,         -- Public URL
  thumbnail_url TEXT,            -- Smaller version

  -- Compression Metrics
  original_size_mb DECIMAL(10,2),
  compressed_size_mb DECIMAL(10,2),
  compression_ratio DECIMAL(5,2), -- e.g., 0.30 = 30% of original

  -- Context
  date_taken DATE,
  location TEXT,
  description TEXT,
  tags TEXT[], -- PostgreSQL array

  -- AI-Generated Metadata
  ai_detected_faces JSONB, -- [{member_id: "uuid", confidence: 0.95, x: 100, y: 150}]
  ai_description TEXT,      -- "Wedding ceremony in garden"
  ai_objects TEXT[],        -- ["wedding", "garden", "celebration"]
  ai_processed BOOLEAN DEFAULT FALSE,

  -- Tracking
  uploaded_by TEXT,
  upload_date TIMESTAMPTZ DEFAULT NOW(),

  -- Search
  search_vector tsvector GENERATED ALWAYS AS (
    to_tsvector('english',
      coalesce(filename, '') || ' ' ||
      coalesce(description, '') || ' ' ||
      coalesce(ai_description, '')
    )
  ) STORED
);

CREATE INDEX idx_media_file_type ON media(file_type);
CREATE INDEX idx_media_date_taken ON media(date_taken);
CREATE INDEX idx_media_tags ON media USING GIN(tags);
CREATE INDEX idx_media_search ON media USING GIN(search_vector);
CREATE INDEX idx_media_ai_processed ON media(ai_processed);
```

Table: media_members (Junction Table)

```
CREATE TABLE media_members (
  media_id UUID REFERENCES media(id) ON DELETE CASCADE,
  member_id UUID REFERENCES family_members(id) ON DELETE CASCADE,

  -- Face location in image (for photos)
  face_coordinates JSONB, -- {x: 100, y: 150, width: 50, height: 50}

  -- Metadata
  tagged_by TEXT,
  tagged_date TIMESTAMPTZ DEFAULT NOW(),

  PRIMARY KEY (media_id, member_id)
);

CREATE INDEX idx_media_members_media ON media_members(media_id);
CREATE INDEX idx_media_members_member ON media_members(member_id);
```

Table: stories

```
CREATE TABLE stories (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- Content
  title TEXT NOT NULL,
  story_text TEXT NOT NULL,
  story_type TEXT CHECK (story_type IN (
    'Life Event', 'Tradition', 'Lesson', 'Hardship',
    'Achievement', 'Humor', 'Migration', 'Other'
  )) NOT NULL,

  -- Context
  storyteller TEXT,
  event_date DATE,
  location TEXT,
  themes TEXT[], -- {resilience, family_unity, migration, achievement}
  language TEXT DEFAULT 'English',

  -- Audio Support
  audio_url TEXT,
  transcribed BOOLEAN DEFAULT FALSE,

  -- AI Enhancements
  ai_summary TEXT,          -- 2-sentence summary
  ai_themes TEXT[],         -- AI-detected themes
  ai_sentiment TEXT CHECK (ai_sentiment IN ('Positive', 'Negative', 'Neutral', 'Mixed')),
  ai_processed BOOLEAN DEFAULT FALSE,
```

```
  -- Metadata
  added_by TEXT,
  added_date TIMESTAMPTZ DEFAULT NOW(),

  -- Search
  search_vector tsvector GENERATED ALWAYS AS (
    to_tsvector('english',
      coalesce(title, '') || ' ' ||
      coalesce(story_text, '') || ' ' ||
      coalesce(ai_summary, '')
    )
  ) STORED,

  -- Constraints
  CONSTRAINT story_text_min_length CHECK (char_length(story_text) >= 50)
);

CREATE INDEX idx_stories_type ON stories(story_type);
CREATE INDEX idx_stories_themes ON stories USING GIN(themes);
CREATE INDEX idx_stories_search ON stories USING GIN(search_vector);
CREATE INDEX idx_stories_event_date ON stories(event_date);
CREATE INDEX idx_stories_transcribed ON stories(transcribed);
```

Table: story_members (Junction Table)

```
CREATE TABLE story_members (
  story_id UUID REFERENCES stories(id) ON DELETE CASCADE,
  member_id UUID REFERENCES family_members(id) ON DELETE CASCADE,

  -- Role in story
  role TEXT CHECK (role IN ('Subject', 'Storyteller', 'Mentioned')) DEFAULT 'Subject',

  PRIMARY KEY (story_id, member_id, role)
);

CREATE INDEX idx_story_members_story ON story_members(story_id);
CREATE INDEX idx_story_members_member ON story_members(member_id);
```

Table: events

```
CREATE TABLE events (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- Basic Info
  name TEXT NOT NULL,
  event_type TEXT CHECK (event_type IN (
    'Wedding', 'Birth', 'Death', 'Graduation', 'Festival',
    'Reunion', 'Achievement', 'Migration', 'Business', 'Other'
  )) NOT NULL,
  event_date DATE NOT NULL,
  location TEXT,

  -- Details
  description TEXT,
  cultural_significance TEXT,

  -- Metadata
  added_by TEXT,
  added_date TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_events_date ON events(event_date DESC);
CREATE INDEX idx_events_type ON events(event_type);
CREATE INDEX idx_events_location ON events(location);
```

Table: event_attendees (Junction Table)

```
CREATE TABLE event_attendees (
  event_id UUID REFERENCES events(id) ON DELETE CASCADE,
  member_id UUID REFERENCES family_members(id) ON DELETE CASCADE,

  -- Role at event
  role TEXT CHECK (role IN ('Host', 'Attendee', 'Subject', 'Organizer')) DEFAULT 'Attendee',

  PRIMARY KEY (event_id, member_id)
);

CREATE INDEX idx_event_attendees_event ON event_attendees(event_id);
CREATE INDEX idx_event_attendees_member ON event_attendees(member_id);
```

Table: traditions

```
CREATE TABLE traditions (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- Basic Info
  name TEXT NOT NULL,
  tradition_type TEXT CHECK (tradition_type IN (
    'Festival', 'Ritual', 'Recipe', 'Custom', 'Language', 'Art', 'Music', 'Other'
  )) NOT NULL,
  description TEXT NOT NULL,
```

```
  -- History
  origin_story TEXT,
  regional_origin TEXT,
  evolution_notes TEXT,

  -- Practice
  frequency TEXT CHECK (frequency IN ('Daily', 'Weekly', 'Monthly', 'Yearly', 'Occasional')),
  still_practiced BOOLEAN DEFAULT TRUE,

  -- Metadata
  added_by TEXT,
  added_date TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_traditions_type ON traditions(tradition_type);
CREATE INDEX idx_traditions_still_practiced ON traditions(still_practiced);
```

Table: tradition_practitioners (Junction Table)

```
CREATE TABLE tradition_practitioners (
  tradition_id UUID REFERENCES traditions(id) ON DELETE CASCADE,
  member_id UUID REFERENCES family_members(id) ON DELETE CASCADE,

  -- Time period they practiced
  time_period TEXT, -- e.g., "1950s-1980s"
  notes TEXT,

  PRIMARY KEY (tradition_id, member_id)
);
```

Table: occupations

```
CREATE TABLE occupations (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- Occupation Info
  occupation_name TEXT NOT NULL UNIQUE,
  category TEXT CHECK (category IN (
    'Agriculture', 'Business', 'Education', 'Government',
    'Healthcare', 'Military', 'Arts', 'Technology',
    'Manufacturing', 'Service', 'Other'
  )),

  -- Notes
  notes TEXT,

  -- Metadata
  created_date TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_occupations_category ON occupations(category);
```

Table: member_occupations (Junction Table with Timeline)

```
CREATE TABLE member_occupations (
  member_id UUID REFERENCES family_members(id) ON DELETE CASCADE,
  occupation_id UUID REFERENCES occupations(id) ON DELETE CASCADE,

  -- Timeline
  start_year INTEGER,
  end_year INTEGER,

  -- Details
  job_title TEXT, -- Specific title if different from occupation name
  organization TEXT,
  notes TEXT, -- "Founded family business", "Promoted to manager"

  PRIMARY KEY (member_id, occupation_id, start_year),

  CONSTRAINT valid_occupation_years CHECK (
    end_year IS NULL OR start_year IS NULL OR end_year >= start_year
  )
);

CREATE INDEX idx_member_occupations_member ON member_occupations(member_id);
CREATE INDEX idx_member_occupations_occupation ON member_occupations(occupation_id);
```

Table: education

```
CREATE TABLE education (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- Student
  member_id UUID NOT NULL REFERENCES family_members(id) ON DELETE CASCADE,

  -- Degree
  degree TEXT NOT NULL, -- "B.Tech Computer Science"
  education_level TEXT CHECK (education_level IN (
    'Primary', 'High School', 'Diploma', 'Undergraduate',
    'Graduate', 'Doctorate', 'Professional', 'Other'
```

```
  )) NOT NULL,

  -- Institution
  institution_name TEXT,
  location TEXT,
  year_completed INTEGER,

  -- Field
  field_of_study TEXT,

  -- Honors
  honors TEXT, -- "First Class", "Gold Medal", "Scholarship"

  -- Metadata
  added_date TIMESTAMPTZ DEFAULT NOW(),

  CONSTRAINT valid_completion_year CHECK (
    year_completed IS NULL OR
    (year_completed >= 1900 AND year_completed <= EXTRACT(YEAR FROM CURRENT_DATE) + 10)
  )
);

CREATE INDEX idx_education_member ON education(member_id);
CREATE INDEX idx_education_level ON education(education_level);
CREATE INDEX idx_education_year ON education(year_completed);
```

Table: locations

```
CREATE TABLE locations (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- Location Info
  name TEXT NOT NULL, -- "Raipur, Chhattisgarh, India"
  location_type TEXT CHECK (location_type IN (
    'Birthplace', 'Residence', 'Event Location', 'Ancestral Village', 'Migration Destination'
  )) NOT NULL,

  -- Coordinates (for mapping)
  latitude DECIMAL(9,6),
  longitude DECIMAL(9,6),

  -- Context
  time_period TEXT, -- "1950s-1980s"
  significance TEXT,
  current_status TEXT CHECK (current_status IN (
    'Family Present', 'Visited Occasionally', 'Lost Contact', 'Sold/Left'
  )),

  -- Metadata
  added_date TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_locations_type ON locations(location_type);
CREATE INDEX idx_locations_coordinates ON locations(latitude, longitude);
```

Table: member_locations (Junction Table with Timeline)

```
CREATE TABLE member_locations (
  member_id UUID REFERENCES family_members(id) ON DELETE CASCADE,
  location_id UUID REFERENCES locations(id) ON DELETE CASCADE,

  -- Relationship
  relationship TEXT CHECK (relationship IN (
    'Born', 'Lived', 'Migrated From', 'Migrated To', 'Visited', 'Worked'
  )) NOT NULL,

  -- Timeline
  start_year INTEGER,
  end_year INTEGER,

  -- Notes
  notes TEXT,

  PRIMARY KEY (member_id, location_id, relationship)
);
```

Table: inbox (Submission Queue)

```
CREATE TABLE inbox (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- Submission Info
  submission_type TEXT CHECK (submission_type IN (
    'New Member', 'Story', 'Media', 'Event', 'Tradition',
    'Update Member', 'Other'
  )) NOT NULL,

  status TEXT CHECK (status IN (
    'Pending', 'Approved', 'Rejected', 'Needs Info', 'Duplicate'
  )) DEFAULT 'Pending',

  -- Data
  raw_data JSONB NOT NULL, -- All form fields as JSON
```

```
  -- Submitter
  submitter_name TEXT NOT NULL,
  submitter_email TEXT,
  submitter_phone TEXT,
  submission_date TIMESTAMPTZ DEFAULT NOW(),

  -- Review
  reviewed_by TEXT,
  review_date TIMESTAMPTZ,
  review_notes TEXT,

  -- Linking (after approval)
  linked_record_id UUID,
  linked_record_type TEXT, -- "family_member", "story", etc.

  -- Attachments (temporary, before moving to R2)
  temp_file_urls TEXT[]
);

CREATE INDEX idx_inbox_status ON inbox(status);
CREATE INDEX idx_inbox_submission_date ON inbox(submission_date DESC);
CREATE INDEX idx_inbox_type ON inbox(submission_type);
```

Table: change_log (Audit Trail)

```
CREATE TABLE change_log (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- What Changed
  table_name TEXT NOT NULL,
  record_id UUID NOT NULL,
  field_name TEXT NOT NULL,

  -- Change Details
  old_value TEXT,
  new_value TEXT,

  -- Who & When
  changed_by TEXT NOT NULL,
  changed_at TIMESTAMPTZ DEFAULT NOW(),

  -- Why
  reason TEXT
);

CREATE INDEX idx_change_log_record ON change_log(table_name, record_id);
CREATE INDEX idx_change_log_date ON change_log(changed_at DESC);
CREATE INDEX idx_change_log_changed_by ON change_log(changed_by);
```

Table: disputes (Conflict Resolution)

```
CREATE TABLE disputes (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  -- What's Disputed
  record_id UUID NOT NULL,
  record_type TEXT NOT NULL, -- "family_member", "story", etc.
  field_name TEXT NOT NULL,

  -- Conflicting Values
  value_a TEXT,
  source_a TEXT, -- "Birth certificate", "Family memory (Uncle Ram)"
  submitted_by_a TEXT,

  value_b TEXT,
  source_b TEXT,
  submitted_by_b TEXT,

  -- Resolution
  resolution TEXT, -- Which value was chosen
  resolution_rationale TEXT,
  resolved_by TEXT,
  resolved_date TIMESTAMPTZ,

  -- Status
  status TEXT CHECK (status IN ('Open', 'Resolved', 'Needs Verification')) DEFAULT 'Open',

  -- Metadata
  created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_disputes_status ON disputes(status);
CREATE INDEX idx_disputes_record ON disputes(record_type, record_id);
```

5.3 Database Views (Pre-computed Queries)

```
-- View: Complete Member Info
CREATE VIEW view_members_complete AS
SELECT
  fm.id,
  fm.full_name,
  fm.nickname,
```

```
  fm.date_of_birth,
  fm.date_of_death,
  fm.status,
  fm.generation,
  fm.lineage,

  -- Computed Age
  get_age(fm.date_of_birth, fm.date_of_death) AS age,

  -- Parents
  p1.full_name AS parent1_name,
  p2.full_name AS parent2_name,

  -- Counts
  (SELECT COUNT(*) FROM marriages WHERE spouse1_id = fm.id OR spouse2_id = fm.id) AS marriage_count,
  (SELECT COUNT(*) FROM family_members WHERE parent1_id = fm.id OR parent2_id = fm.id) AS children_count,
  (SELECT COUNT(*) FROM media_members WHERE member_id = fm.id) AS media_count,
  (SELECT COUNT(*) FROM story_members WHERE member_id = fm.id) AS story_count,

  -- Latest Photo
  (SELECT r2_url FROM media m
   JOIN media_members mm ON m.id = mm.media_id
   WHERE mm.member_id = fm.id AND m.file_type = 'Photo'
   ORDER BY m.date_taken DESC NULLS LAST, m.upload_date DESC
   LIMIT 1) AS latest_photo_url

FROM family_members fm
LEFT JOIN family_members p1 ON fm.parent1_id = p1.id
LEFT JOIN family_members p2 ON fm.parent2_id = p2.id;

-- View: Occupation Trends
CREATE VIEW view_occupation_trends AS
SELECT
  o.occupation_name,
  o.category,
  COUNT(DISTINCT mo.member_id) AS member_count,
  MIN(mo.start_year) AS first_year,
  MAX(mo.end_year) AS last_year,
  ARRAY_AGG(DISTINCT fm.generation ORDER BY fm.generation) AS generations_practiced
FROM occupations o
JOIN member_occupations mo ON o.id = mo.occupation_id
JOIN family_members fm ON mo.member_id = fm.id
GROUP BY o.id, o.occupation_name, o.category
ORDER BY member_count DESC;

-- View: Migration Timeline
CREATE VIEW view_migration_timeline AS
SELECT
  fm.full_name,
  fm.date_of_birth,
  l.name AS location_name,
  ml.relationship,
  ml.start_year,
  ml.end_year,
  l.latitude,
  l.longitude
FROM member_locations ml
JOIN family_members fm ON ml.member_id = fm.id
JOIN locations l ON ml.location_id = l.id
WHERE ml.relationship IN ('Migrated From', 'Migrated To', 'Born', 'Lived')
ORDER BY fm.generation, fm.date_of_birth, ml.start_year;

-- View: Statistics Dashboard
CREATE VIEW view_statistics AS
SELECT
  (SELECT COUNT(*) FROM family_members) AS total_members,
  (SELECT COUNT(*) FROM family_members WHERE status = 'Living') AS living_members,
  (SELECT COUNT(*) FROM family_members WHERE status = 'Deceased') AS deceased_members,
  (SELECT COUNT(*) FROM media) AS total_media,
  (SELECT COUNT(*) FROM stories) AS total_stories,
  (SELECT COUNT(*) FROM events) AS total_events,
  (SELECT MAX(generation) FROM family_members) AS max_generation,
  (SELECT AVG(get_age(date_of_birth, date_of_death))
   FROM family_members
   WHERE status = 'Deceased' AND date_of_birth IS NOT NULL AND date_of_death IS NOT NULL) AS avg_lifespan,
  (SELECT mode() WITHIN GROUP (ORDER BY gender) FROM family_members WHERE gender IS NOT NULL) AS most_common_gender,
  (SELECT COUNT(DISTINCT member_id) FROM education WHERE education_level IN ('Undergraduate', 'Graduate', 'Doctorate')) AS college_educated_count;
```

5.4 Row-Level Security (RLS) Policies

```
-- Enable RLS on all tables
ALTER TABLE family_members ENABLE ROW LEVEL SECURITY;
ALTER TABLE media ENABLE ROW LEVEL SECURITY;
ALTER TABLE stories ENABLE ROW LEVEL SECURITY;
-- ... repeat for all tables

-- Policy: Anyone authenticated can view
CREATE POLICY "Authenticated users can view all data"
ON family_members FOR SELECT
TO authenticated
USING (true);

-- Policy: Only admins can insert/update/delete
CREATE POLICY "Only admins can modify data"
ON family_members FOR ALL
TO authenticated
USING (
```

```
  auth.jwt() ->> 'email' IN (
    SELECT email FROM admin_users
  )
)
WITH CHECK (
  auth.jwt() ->> 'email' IN (
    SELECT email FROM admin_users
  )
);

-- Policy: Contact info only for admins
CREATE POLICY "Contact info private"
ON family_members FOR SELECT
TO authenticated
USING (
  CASE
    WHEN auth.jwt() ->> 'email' IN (SELECT email FROM admin_users)
    THEN true
    ELSE (contact_info IS NULL)
  END
);

-- Admin Users Table
CREATE TABLE admin_users (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  email TEXT UNIQUE NOT NULL,
  role TEXT CHECK (role IN ('Super Admin', 'Admin', 'Moderator')) DEFAULT 'Admin',
  added_by TEXT,
  added_date TIMESTAMPTZ DEFAULT NOW()
);

-- Insert initial admin
INSERT INTO admin_users (email, role, added_by)
VALUES ('your-email@example.com', 'Super Admin', 'System');
```

5.5 Database Functions (Stored Procedures)

```
-- Function: Get All Descendants
CREATE OR REPLACE FUNCTION get_descendants(ancestor_id UUID)
RETURNS TABLE (
  id UUID,
  full_name TEXT,
  generation INTEGER,
  relationship TEXT
) AS $$
WITH RECURSIVE descendants AS (
  -- Base case: children of ancestor
  SELECT
    id,
    full_name,
    generation,
    'Child' AS relationship,
    1 AS depth
  FROM family_members
  WHERE parent1_id = ancestor_id OR parent2_id = ancestor_id

  UNION ALL

  -- Recursive case: grandchildren, great-grandchildren, etc.
  SELECT
    fm.id,
    fm.full_name,
    fm.generation,
    CASE
      WHEN d.depth = 1 THEN 'Grandchild'
      WHEN d.depth = 2 THEN 'Great-Grandchild'
      ELSE 'Descendant (Gen +' || (d.depth + 1) || ')'
    END,
    d.depth + 1
  FROM family_members fm
  INNER JOIN descendants d ON (fm.parent1_id = d.id OR fm.parent2_id = d.id)
)
SELECT id, full_name, generation, relationship FROM descendants;
$$ LANGUAGE SQL STABLE;

-- Function: Get All Ancestors
CREATE OR REPLACE FUNCTION get_ancestors(descendant_id UUID)
RETURNS TABLE (
  id UUID,
  full_name TEXT,
  generation INTEGER,
  relationship TEXT
) AS $$
WITH RECURSIVE ancestors AS (
  -- Base case: parents
  SELECT
    id,
    full_name,
    generation,
    'Parent' AS relationship,
    1 AS depth
  FROM family_members
  WHERE id IN (
    SELECT parent1_id FROM family_members WHERE id = descendant_id
    UNION
    SELECT parent2_id FROM family_members WHERE id = descendant_id
  )
```

```
  UNION ALL

  -- Recursive case: grandparents, great-grandparents, etc.
  SELECT
    fm.id,
    fm.full_name,
    fm.generation,
    CASE
      WHEN a.depth = 1 THEN 'Grandparent'
      WHEN a.depth = 2 THEN 'Great-Grandparent'
      ELSE 'Ancestor (Gen -' || (a.depth + 1) || ')'
    END,
    a.depth + 1
  FROM family_members fm
  INNER JOIN ancestors a ON (fm.id = a.parent1_id OR fm.id = a.parent2_id)
  WHERE fm.id IN (
    SELECT parent1_id FROM family_members WHERE id = a.id
    UNION
    SELECT parent2_id FROM family_members WHERE id = a.id
  )
)
SELECT id, full_name, generation, relationship FROM ancestors;
$$ LANGUAGE SQL STABLE;

-- Function: Calculate Generation
CREATE OR REPLACE FUNCTION calculate_generation(member_id UUID)
RETURNS INTEGER AS $$
DECLARE
  parent1_gen INTEGER;
  parent2_gen INTEGER;
  max_parent_gen INTEGER;
BEGIN
  -- Get parent generations
  SELECT generation INTO parent1_gen FROM family_members WHERE id = (
    SELECT parent1_id FROM family_members WHERE id = member_id
  );

  SELECT generation INTO parent2_gen FROM family_members WHERE id = (
    SELECT parent2_id FROM family_members WHERE id = member_id
  );

  -- Return max parent generation + 1
  max_parent_gen := GREATEST(
    COALESCE(parent1_gen, 0),
    COALESCE(parent2_gen, 0)
  );

  RETURN max_parent_gen + 1;
END;
$$ LANGUAGE plpgsql STABLE;

-- Trigger: Auto-calculate generation on insert/update
CREATE OR REPLACE FUNCTION set_generation()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.generation IS NULL OR OLD.parent1_id != NEW.parent1_id OR OLD.parent2_id != NEW.parent2_id THEN
    NEW.generation := calculate_generation(NEW.id);
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_set_generation
BEFORE INSERT OR UPDATE ON family_members
FOR EACH ROW EXECUTE FUNCTION set_generation();

-- Function: Search Stories by Semantic Similarity
CREATE OR REPLACE FUNCTION search_stories_semantic(
  query_embedding vector(768),
  match_threshold float,
  match_count int
)
RETURNS TABLE (
  id uuid,
  title text,
  story_text text,
  similarity float
) AS $$
BEGIN
  RETURN QUERY
  SELECT
    stories.id,
    stories.title,
    stories.story_text,
    1 - (stories.embedding <=> query_embedding) AS similarity
  FROM stories
  WHERE 1 - (stories.embedding <=> query_embedding) > match_threshold
  ORDER BY stories.embedding <=> query_embedding
  LIMIT match_count;
END;
$$ LANGUAGE plpgsql;
```

5.6 Database Optimization Strategy

Indexing Strategy

✅ Index all foreign keys ✅ Index frequently queried fields (name, date, status) ✅ GIN indexes for full-text search, arrays, JSONB ✅ Partial indexes for common filters (e.g., WHERE status = 'Living')

Query Optimization

✅ Use views for complex joined queries ✅ Materialized views for expensive aggregations (refresh periodically) ✅ EXPLAIN ANALYZE to identify slow queries ✅ Connection pooling via Supabase (PgBouncer)

Data Archiving

🗓️ Year 5+: Move deceased members from >1900 to archive table 🗓️ Year 10+: Partition tables by generation or decade

Performance Monitoring

```
-- Query to find slow queries
SELECT
  query,
  calls,
  total_time,
  mean_time,
  max_time
FROM pg_stat_statements
ORDER BY mean_time DESC
LIMIT 10;

-- Query to find missing indexes
SELECT
  schemaname,
  tablename,
  attname,
  n_distinct,
  correlation
FROM pg_stats
WHERE schemaname = 'public'
AND n_distinct > 100
AND correlation < 0.1;
```

6. FEATURE SPECIFICATIONS

6.1 Feature List (Complete - 96 Features)

CORE FEATURES (Features 1-67)

#	Feature Name	Priority	Category	Status
1	Family Tree Visualization	P0	Core	Week 2
2	Member Profiles	P0	Core	Week 2
3	Media Gallery	P0	Core	Week 3
4	Story Archive	P0	Core	Week 4
5	Events Timeline	P0	Core	Week 4
6	Admin Dashboard	P0	Admin	Week 1
7	Submission Inbox	P0	Admin	Week 7
8	Search System	P0	Discovery	Week 5
9	Mobile Upload	P0	Upload	Week 8
10	Auto-Compression	P0	Media	Week 5
11	Smart Image Tagging	P1	AI	Week 5
12	Face Detection	P1	AI	Week 6
13	Story Theme Extraction	P1	AI	Week 6
14	Auto-Summarization	P1	AI	Week 6
15	Audio Transcription	P1	AI	Week 10
16	Semantic Search	P1	AI	Week 6
17	Duplicate Detection	P1	AI	Week 7
18	Smart Captions	P1	AI	Week 5
19	Family Statistics Dashboard	P1	Analytics	Week 9
20	Occupation Trends	P1	Analytics	Week 9
21	Education Evolution	P1	Analytics	Week 9
22	Migration Map	P1	Analytics	Week 9
23	Generation Comparison	P1	Analytics	Week 9
24	Firsts & Achievements	P1	Analytics	Week 9
25	Timeline View	P1	Visualization	Week 4
26	Data Completeness Tracker	P1	Admin	Week 12
27	Public Submission Forms	P0	Collaboration	Week 7
28	Contributor Leaderboard	P2	Collaboration	Week 11
29	Weekly Prompts	P2	Collaboration	Week 11
30	Member Spotlight	P2	Collaboration	Week 11
31	Comment System	P2	Collaboration	Week 12
32	Verification Workflow	P2	Collaboration	Week 12
33	Notification System	P1	Engagement	Week 8
34	Family News Feed	P2	Engagement	Week 8
35	Birthday Reminders	P1	Reminders	Week 11

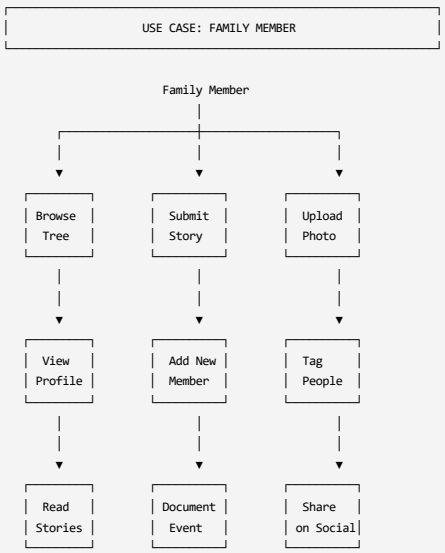
#	Feature Name	Priority	Category	Status
36	Anniversary Alerts	P2	Reminders	Week 11
37	Festival Calendar	P2	Reminders	Week 11
38	Milestone Tracker	P2	Reminders	Week 11
39	Tags System	P1	Organization	Week 3
40	Collections	P2	Organization	Week 12
41	Privacy Controls	P0	Security	Week 2
42	Version History	P1	Security	Week 12
43	Bulk Operations	P2	Admin	Week 12
44	Advanced Filters	P1	Discovery	Week 5
45	Tradition Documentation	P1	Culture	Week 11
46	Recipe Book	P2	Culture	Week 11
47	Document Vault	P2	Documents	Week 10
48	Occupation Directory	P1	Genealogy	Week 9
49	Location Registry	P1	Genealogy	Week 9
50	Relationship Timeline	P2	Genealogy	Week 12
51	GEDCOM Export	P1	Export	Week 12
52	PDF Reports	P1	Export	Week 12
53	Social Sharing	P2	Sharing	Future
54	Public Family Website	P3	Sharing	Future
55	Data Backup	P0	Backup	Week 12
56	Collaborative Editing	P1	Admin	Week 2
57	Role-Based Access	P0	Security	Week 2
58	Audit Trail	P1	Security	Week 12
59	Encrypted Storage	P1	Security	Week 2
60	Two-Factor Authentication	P1	Security	Week 2
61	Data Retention Policies	P2	Admin	Week 12
62	GDPR Compliance	P2	Legal	Week 12
63	API Access	P3	Integration	Future
64	Webhooks	P3	Integration	Future
65	Multi-Language Support	P2	I18n	Future
66	Voice Interface	P4	Experimental	Future
67	AR/VR Support	P4	Experimental	Far Future

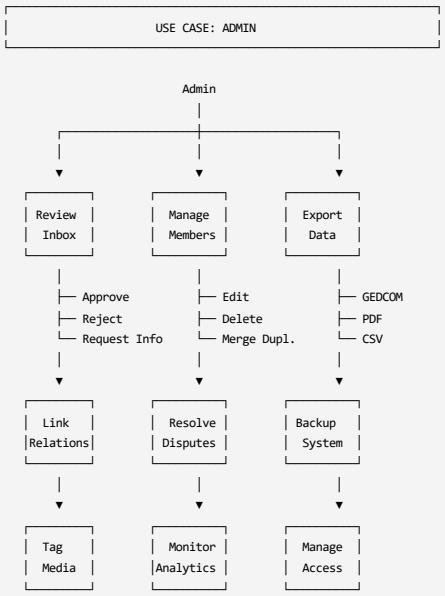
NEW FEATURES (Features 68-96)

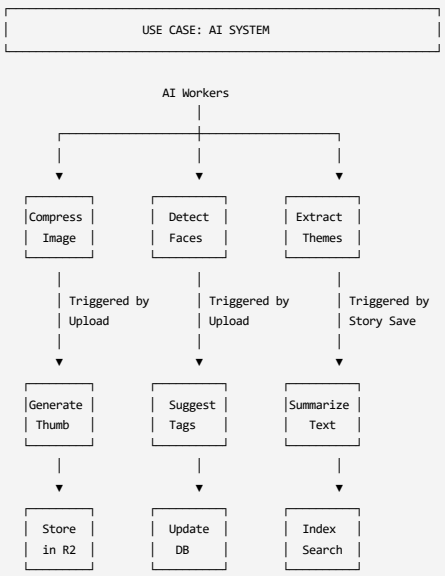
#	Feature Name	Priority	Implementation Timeline
68	DNA/Genetic Heritage Integration	P1	Months 7-9
69	WhatsApp Bot for Submissions	P1	Months 7-9
70	Collaborative Family Book Generator	P1	Months 10-12
71	Smart Relationship Suggester	P1	Months 7-9
72	Video Story Recording Booth	P1	Months 10-12
73	Health/Medical History Tracker	P2	Year 2 Q1
74	Property/Asset Timeline	P2	Year 2 Q1
75	Multilingual AI Translation	P2	Year 2 Q2
76	Face Recognition Auto-Tagging	P2	Year 2 Q2
77	Cultural Glossary	P2	Year 2 Q3
78	Personality Traits Archive	P2	Year 2 Q3
79	Family Name Etymology	P2	Year 2 Q4
80	Historical Context Layer	P3	Year 3+
81	Ancestry Match Finder	P3	Year 3+
82	Skills & Talents Registry	P3	Year 3+
83	Memorial Pages	P3	Year 3+
84	Family Crest/Logo Generator	P3	Year 3+
85	Heirloom Tracker	P3	Year 3+
86	Family Quiz Generator	P3	Year 3+
87	Predictive Analytics	P3	Year 3+
88	Social Network Graph	P3	Year 3+
89	Life Comparison Tool	P3	Year 3+
90	AI Ancestor Voice Synthesis	P4	Evaluate demand
91	Deepfake Photo Animation	P4	Evaluate demand
92	AI Chat with Ancestors	P4	Evaluate demand
93	Genetic Time Machine	P4	Evaluate demand
94	Family Metaverse Space	P4	2030+
95	Blockchain Heritage Certificate	P4	Evaluate demand
96	AI Life Advice from Ancestors	P4	Evaluate demand

Total: 96 Features

6.2 Use Case Diagrams







6.3 User Stories (Detailed)

Epic 1: Data Collection

US-001: Submit Family Member

- **As a** family member
- **I want to** add a new person to the family tree
- **So that** their information is preserved

Acceptance Criteria:

- Form has fields: name, nickname, gender, DOB, birth place
- Can select parents from dropdown (existing members)
- Can add spouse(s)
- Can mark as living or deceased
- Form validates required fields
- Duplicate name warning shown
- Submission goes to inbox for admin review
- Confirmation email sent

US-002: Upload Photos

- **As a** family member
- **I want to** upload family photos from my phone
- **So that** memories are shared with everyone

Acceptance Criteria:

- Can select multiple photos at once
- Can use camera to take new photo
- Can add description
- Can tag people in photo
- Can specify date and location
- Photos automatically compressed
- Progress bar shows upload status
- Thumbnail generated automatically

US-003: Share a Story

- **As an** elder family member
- **I want to** record a story about the past
- **So that** younger generations learn our history

Acceptance Criteria:

- Can write text story (min 50 chars)
- Can upload audio recording
- Can tag which family members are in story
- Can categorize (achievement, hardship, humor, etc.)
- Can specify when/where it happened
- Can add themes (resilience, family unity, etc.)
- Audio auto-transcribed (if applicable)
- Story appears in relevant member profiles

Epic 2: Data Discovery

US-010: Browse Family Tree

- **As a** family member
- **I want to** explore the family tree visually
- **So that** I understand how everyone is related

Acceptance Criteria:

- Tree shows all members with photos
- Can zoom in/out
- Can click member to see profile
- Can filter by lineage (father's/mother's side)
- Can filter by generation
- Can filter by living/deceased
- Can search by name
- Tree auto-arranges (no manual positioning)

US-011: Search Everything

- **As a** researcher
- **I want to** search across all family data
- **So that** I can find specific information quickly

Acceptance Criteria:

- Single search box searches: names, stories, events, media
- Shows results grouped by type
- Can filter results by date range
- Can filter by person
- Semantic search (finds meaning, not just keywords)
- Search highlights matched terms
- Search history saved

Epic 3: Admin Operations

US-020: Review Submissions

- **As an** admin
- **I want to** review family submissions before approval
- **So that** data quality is maintained

Acceptance Criteria:

- Inbox shows pending submissions
- Can view full submission data
- Can approve (creates record in main DB)
- Can reject (sends email to submitter with reason)
- Can request more info
- Can detect duplicates (fuzzy name match)
- Can link to existing members
- Can bulk approve multiple submissions

US-021: Resolve Data Conflicts

- **As an** admin
- **I want to** handle disputes about facts
- **So that** accurate information is preserved

Acceptance Criteria:

- System flags conflicting data
- Can see both versions side-by-side
- Can see sources for each version
- Can choose resolution
- Can mark as "unverified" if uncertain
- Resolution logged in audit trail
- Submitters notified of decision

Epic 4: AI Features

US-030: Auto-Tag Photos

- **As a** user
- **I want** photos automatically tagged
- **So that** I don't have to manually tag thousands of photos

Acceptance Criteria:

- AI detects faces in photo
- AI suggests which family member
- User confirms or corrects
- AI learns from corrections
- Tags added to photo metadata
- Photo searchable by tagged people

US-031: Transcribe Audio Stories

- **As an** admin
- **I want** audio recordings transcribed automatically
- **So that** stories are searchable

Acceptance Criteria:

- Upload audio file
- AI converts speech to text
- Transcript saved with story
- Can edit transcript if errors
- Transcript searchable
- Original audio file preserved

6.4 Functional Requirements (Detailed)

FR-1: Authentication & Authorization

FR-1.1 User Registration

- Users can sign up with email/password
- Email verification required
- Password must meet strength requirements (8+ chars, 1 number, 1 special)

FR-1.2 User Login

- Login with email/password
- "Remember me" option (30-day session)
- Password reset via email

FR-1.3 Role-Based Access Control

- Roles: Super Admin, Admin, Contributor, Viewer
- Permissions matrix enforced server-side
- RLS policies in Supabase

FR-1.4 Two-Factor Authentication (Optional)

- SMS or authenticator app
- Required for Super Admin
- Optional for other roles

FR-2: Member Management

FR-2.1 Create Member

- Required fields: full_name, lineage
- Optional fields: all others
- Auto-generate UUID
- Auto-calculate generation from parents
- Validate: DOB < DOD, no future dates

FR-2.2 Edit Member

- All fields editable by admin
- Changes logged in change_log table
- Real-time updates via Supabase subscriptions

FR-2.3 Delete Member

- Soft delete (mark as deleted, don't remove)
- Cascade updates relationships
- Requires Super Admin permission

FR-2.4 Link Relationships

- Can add/remove parents
- Can add/remove spouses (via marriages table)
- Can add/remove children (auto-computed from parent links)
- Validates: no circular relationships, no self-references

FR-3: Media Upload & Management

FR-3.1 Upload Flow

```
User selects file
  → Client-side validation (type, size)
  → Upload to temporary location
  → Call /api/compress (for images)
  → Upload compressed to R2
  → Create media record in DB
  → Return URL to client
```

FR-3.2 Compression

- Images: Convert to WebP, 85% quality
- Target max width: 1920px (preserve aspect ratio)
- Generate thumbnail: 400x400px
- Log original vs compressed size

FR-3.3 Video Handling

- Accept: MP4, MOV, AVI
- Transcode to H.264 MP4 (if not already)
- Max resolution: 1080p
- Max duration: 30 minutes
- Extract thumbnail at 0:01

FR-3.4 Audio Handling

- Accept: MP3, WAV, M4A
- Convert to MP3 128kbps
- Max duration: 2 hours

FR-3.5 Document Handling

- Accept: PDF, DOCX, JPG (scanned docs)
- OCR text extraction (for searchability)
- Max size: 50MB

FR-4: Search & Discovery

FR-4.1 Full-Text Search

- Search across: names, stories, media descriptions
- Uses PostgreSQL tsvector
- Ranks by relevance
- Highlights matched terms

FR-4.2 Semantic Search

- Uses text embeddings (Cloudflare AI)
- Finds similar meaning, not just keywords

- Example: "resilience" matches stories about overcoming hardship
- Cosine similarity matching

FR-4.3 Filters

- By person (multi-select)
- By date range
- By location
- By tags
- By media type
- By story type
- Combine multiple filters (AND logic)

FR-4.4 Advanced Query Syntax

- Quotes for exact match: "wedding ceremony"
- OR operator: migration OR relocation
- Exclude: -draft
- Field-specific: name:Ramesh

FR-5: AI Processing

FR-5.1 Image Analysis Pipeline

```
Image uploaded
  → Detect faces (Cloudflare AI)
  → For each face:
      → Extract coordinates
      → Match against known faces (if any)
      → Suggest member_id
  → Detect objects/scenes
  → Generate caption
  → Store in ai_detected_faces, ai_description
  → Update ai_processed flag
```

FR-5.2 Story Analysis Pipeline

```
Story saved
  → Extract themes (LLaMA 2 via Cloudflare)
  → Generate 2-sentence summary
  → Detect sentiment (positive/negative/neutral)
  → Store in ai_summary, ai_themes, ai_sentiment
  → Update ai_processed flag
```

FR-5.3 Audio Transcription Pipeline

```
Audio uploaded
  → Send to Whisper model (Cloudflare AI)
  → Receive transcript
  → Store in story_text
  → Mark transcribed = true
  → Store original audio_url
```

FR-5.4 Duplicate Detection

```
New member submitted
  → Get all existing members
  → Compare name similarity (Levenshtein distance)
  → If similarity > 80% AND DOB within 5 years
      → Flag as possible duplicate
  → Notify admin in inbox
  → Admin decides: merge, keep separate, or reject
```

FR-6: Backup & Export

FR-6.1 Automated Daily Backup

- Runs at 2 AM server time
- Exports all tables to JSON
- Commits to GitHub repository
- Sends email confirmation
- Retains 30 days of backups

FR-6.2 GEDCOM Export

- Generates .ged file (genealogy standard)
- Includes: members, relationships, events
- Download as file
- Compatible with Ancestry, FamilySearch, etc.

FR-6.3 PDF Report Generation

- Template-based (customizable)
- Includes: member profiles, photos, stories
- Pagination, table of contents
- Download or email

FR-6.4 Full Data Export

- ZIP file containing:
  - All database tables as CSV
  - All media files (original folders preserved)
  - README with schema documentation
- One-click download

FR-7: Notifications

FR-7.1 Email Notifications

- New submission received (to admin)
- Submission approved/rejected (to submitter)
- Birthday reminders (to family)
- Weekly digest (to contributors)

FR-7.2 In-App Notifications

- Real-time badge updates
- Notification center
- Mark as read/unread
- Clear all option

FR-7.3 Push Notifications (PWA)

- Birthday reminders
- New family member added
- Someone tagged you in photo
- Your submission approved

FR-8: Analytics & Insights

FR-8.1 Family Statistics

- Total members count
- Living vs deceased ratio
- Average lifespan by generation
- Gender distribution
- Geographic spread

FR-8.2 Occupation Trends

- Most common occupations
- Career transitions over generations
- First person in each occupation
- Occupation categories pie chart

FR-8.3 Education Evolution

- Education levels by generation
- First college graduate
- Fields of study trends
- Institutions attended map

FR-8.4 Migration Patterns

- Interactive map with migration routes
- Timeline of migrations
- Push/pull factors (if documented)
- Current family distribution

FR-8.5 Data Completeness

- % complete profiles
- Missing data report
- Fields most often incomplete
- Suggestions for data collection

FR-9: Collaboration Features

FR-9.1 Comments

- Can comment on: members, stories, media, events
- Threaded replies
- Mentions (@username)
- Edit/delete own comments
- Admin can moderate

FR-9.2 Reactions

- Emoji reactions on stories/media
- Like, love, celebrate, etc.
- Count displayed
- Who reacted (visible to all)

FR-9.3 Contributor Badges

- Auto-assigned based on contributions
- Historian: 10+ stories
- Archivist: 50+ photos
- Connector: 20+ relationships linked
- Displayed on profile

FR-9.4 Activity Feed

- Recent actions by family members
- Filterable by person or type
- "Mark all as read"
- Pagination (show more)

FR-10: Privacy & Permissions

FR-10.1 Visibility Controls

- Public: Anyone can see
- Family Only: Authenticated users
- Admins Only: Restricted
- Private: Only creator

FR-10.2 Living Member Privacy

- Contact info hidden by default
- Can opt-out of public sharing
- Request data deletion (GDPR)

FR-10.3 Sensitive Content Flagging

- Stories can be marked "sensitive"
- Warning shown before viewing
- Option to hide from children

FR-10.4 Data Download

- Users can download their contributions
- Export personal data (GDPR)
- Includes: submissions, comments, uploads

FR-11: Mobile-Specific Features

FR-11.1 Offline Mode

- Cache recent data for offline viewing
- Queue submissions when offline
- Sync when connection restored
- Indicator shows offline status

FR-11.2 Camera Integration

- Take photo directly in app
- Use front/back camera
- Apply filters (optional)
- Add caption before upload

FR-11.3 Voice Input

- Dictate story text
- Speech-to-text
- Edit after transcription
- Multiple languages supported

FR-11.4 Share to App

- Share photos from gallery to app
- Share contacts to add member
- Share location for events
- iOS/Android share sheet integration

FR-12: Gamification

FR-12.1 Leaderboard

- Top contributors this month
- All-time contributors
- Category leaderboards (photos, stories, etc.)
- Opt-out option

FR-12.2 Achievements

- Unlock achievements for milestones
- First submission
- 100 photos uploaded
- Complete profile for 10 members
- Share achievement on feed

FR-12.3 Challenges

- Weekly challenges
- "Share a childhood memory"
- "Upload 5 photos from the 1980s"
- "Document a family recipe"
- Completion badge

FR-13: Advanced Features

FR-13.1 Bulk Import

- Import from CSV file
- Map columns to fields
- Preview before import
- Error handling & validation
- Import log/report

FR-13.2 Merge Duplicates

- Detect duplicate members
- Show side-by-side comparison
- Select which data to keep
- Merge relationships
- Log merge action

FR-13.3 Timeline View

- Chronological view of all events
- Filter by person, event type, year
- Zoom in/out (decade, year, month)
- Add new events inline

FR-13.4 Relationship Calculator

- "How am I related to this person?"
- Shows relationship path
- Common ancestors
- Degree of relationship

FR-14: System Administration

FR-14.1 User Management

- View all users
- Add/remove admin privileges
- Ban/unban users
- View user activity log

FR-14.2 Content Moderation

- Flag inappropriate content
- Review flagged items
- Remove/hide content
- Ban repeat offenders

FR-14.3 System Health

- Database size monitor
- API usage stats
- Storage usage (R2)
- Error logs
- Performance metrics

FR-14.4 Maintenance Mode

- Enable/disable submissions
- Display maintenance message
- Schedule maintenance window
- Notify users in advance

7. USER INTERFACE DESIGN

7.1 Information Architecture

```
┌─────────────────────────────────────────────────┐
│                   SITE MAP                      │
└─────────────────────────────────────────────────┘

Root (/)
│
├── Home / Dashboard
│   ├── Family Statistics Widget
│   ├── Recent Activity Feed
│   ├── Quick Actions
│   └── Birthday Reminders
│
├── Family Tree (/tree)
│   ├── Interactive Tree View
```

```
│   ├── Filter Panel (sidebar)
│   ├── Search Bar
│   └── Legend
│
├── Members (/members)
│   ├── List View (table)
│   ├── Grid View (cards)
│   ├── Filter & Sort Options
│   └── Individual Profile (/members/[id])
│       ├── Bio & Details
│       ├── Relationships Tab
│       ├── Photos Tab
│       ├── Stories Tab
│       ├── Timeline Tab
│       └── Edit Button (admin only)
│
├── Media (/media)
│   ├── Gallery View (photos)
│   ├── Videos Tab
│   ├── Audio Tab
│   ├── Documents Tab
│   ├── Filter Panel
│   ├── Upload Button
│   └── Media Detail (/media/[id])
│       ├── Full-size Display
│       ├── Tagged People
│       ├── Comments
│       └── Download/Share
│
├── Stories (/stories)
│   ├── Card View (previews)
│   ├── Filter by Type
│   ├── Filter by Person
│   ├── Add Story Button
│   └── Story Detail (/stories/[id])
│       ├── Full Text
│       ├── Audio Player (if applicable)
│       ├── Related People
│       ├── Comments
│       └── Share
│
├── Events (/events)
│   ├── Timeline View (default)
│   ├── Calendar View
│   ├── List View
│   ├── Filter Options
│   └── Event Detail (/events/[id])
│       ├── Description
│       ├── Attendees
│       ├── Photos
│       └── Stories
│
├── Traditions (/traditions)
│   ├── Grid View
│   ├── Filter by Type
│   ├── Add Tradition Button
│   └── Tradition Detail (/traditions/[id])
│       ├── Description
│       ├── Origin Story
│       ├── Practitioners
│       └── Media
│
├── Analytics (/analytics)
│   ├── Statistics Dashboard
│   ├── Occupation Trends Chart
│   ├── Education Evolution Chart
│   ├── Migration Map
│   ├── Generation Comparison
│   └── Achievements List
│
├── Admin (/admin)
│   ├── Dashboard
│   │   ├── Inbox Count
│   │   ├── Recent Changes
│   │   ├── Data Quality Metrics
│   │   └── Quick Actions
│   │
│   ├── Inbox (/admin/inbox)
│   │   ├── Pending Tab
│   │   ├── Approved Tab
│   │   ├── Rejected Tab
│   │   └── Submission Detail (/admin/inbox/[id])
│   │       ├── Raw Data View
│   │       ├── Approve/Reject Buttons
│   │       ├── Link to Existing Records
│   │       └── Request More Info
│   │
│   ├── Users (/admin/users)
│   │   ├── User List
│   │   ├── Add User
│   │   └── Manage Roles
│   │
│   ├── Backups (/admin/backups)
│   │   ├── Backup History
│   │   ├── Restore Point
│   │   └── Export Data
│   │
│   └── Settings (/admin/settings)
│       ├── Site Configuration
│       ├── Email Templates
```

```
│       ├── Privacy Settings
│       └── API Keys
│
├── Search (/search)
│   ├── Search Bar (prominent)
│   ├── Results (grouped by type)
│   ├── Filters
│   └── Advanced Search Options
│
├── Submit (/submit)
│   ├── Add Member Form
│   ├── Share Story Form
│   ├── Upload Media Form
│   ├── Document Event Form
│   └── Record Tradition Form
│
└── Account (/account)
    ├── Profile
    ├── Notifications Settings
    ├── Privacy Settings
    └── Logout
```

7.2 Wireframes (ASCII Art)

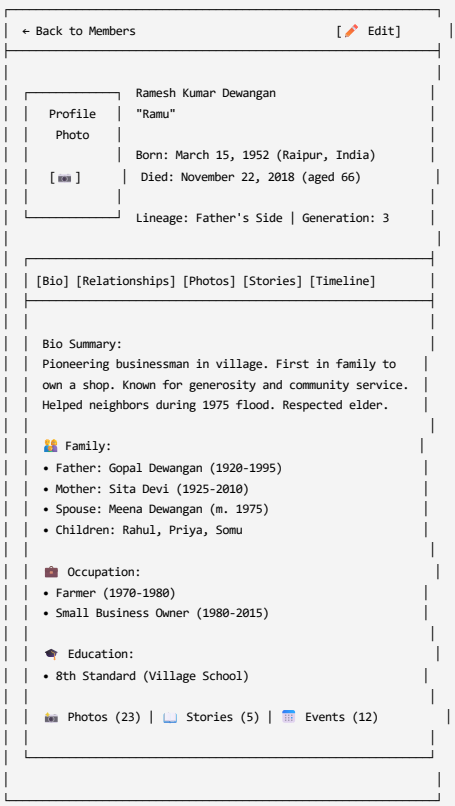
Home / Dashboard

```
┌─────────────────────────────────────────────────────────┐
│ [Logo] Roots of Dewangan              [Search] [👤 Profile] │
├─────────────────────────────────────────────────────────┤
│                                                         │
│  ┌──────────────────┐  ┌──────────────────────────────┐ │
│  │  Family Statistics  │  │     Recent Activity          │ │
│  │                  │  │                              │ │
│  │  👥 Total: 142     │  │  • Priya added 5 photos      │ │
│  │  💚 Living: 89     │  │  • Ram shared a story        │ │
│  │  🕊️ Deceased: 53   │  │  • Admin approved 3 members  │ │
│  │  📸 Photos: 856    │  │  • New event added           │ │
│  │  📖 Stories: 47    │  │  • 2 pending submissions     │ │
│  │                  │  │                              │ │
│  │  [View Full Stats]  │  │  [View All Activity]         │ │
│  └──────────────────┘  └──────────────────────────────┘ │
│                                                         │
│  ┌─────────────────────────────────────────────────┐    │
│  │              Quick Actions                      │    │
│  │                                                 │    │
│  │  [+ Add Member]  [📷 Upload Photo]  [✍️ Share Story]  │    │
│  │                                                 │    │
│  │  [🌳 View Tree]  [📊 Analytics]    [⚙️ Admin Panel]  │    │
│  └─────────────────────────────────────────────────┘    │
│                                                         │
│  ┌─────────────────────────────────────────────────┐    │
│  │           🎂 Upcoming Birthdays                 │    │
│  │                                                 │    │
│  │  Feb 10 - Ramesh Dewangan (turns 72)            │    │
│  │  Feb 15 - Priya Sharma (turns 28)               │    │
│  │  Feb 22 - Somu Dewangan (turns 30)              │    │
│  │                                                 │    │
│  └─────────────────────────────────────────────────┘    │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

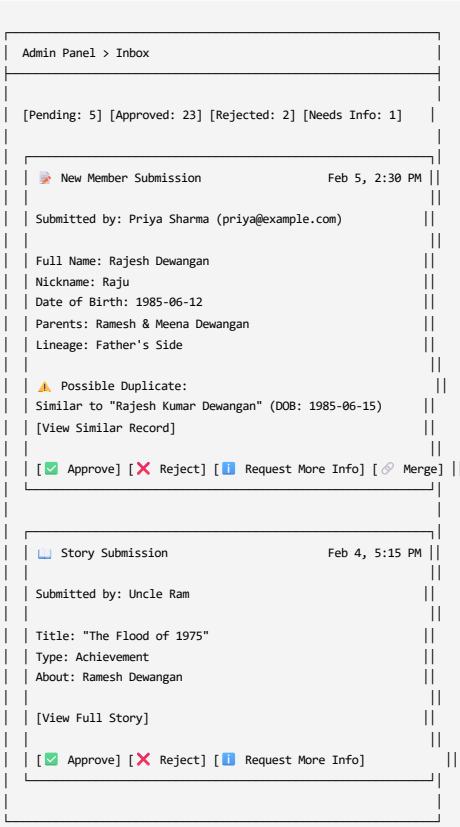
Family Tree View

```
┌─────────────────────────────────────────────────────────┐
│ ← Back to Home          Family Tree          [⚙️ Filter] │
├─────────────────────────────────────────────────────────┤
│                                                         │
│  ┌──────────┐                                           │
│  │ Filters  │                                           │
│  │          │    ┌──────────────────────┐               │
│  │ Lineage: │    │                      │               │
│  │ ☑ Father's │    │   [Gopal Dewangan]   │               │
│  │ ☑ Mother's │    │      (1920-1995)     │               │
│  │          │    │        Gen 1         │               │
│  │ Status:  │    └──────────┬───────────┘               │
│  │ ☑ Living   │               │                           │
│  │ ☑ Deceased │               ├──────────┬──────────┐     │
│  │          │               │          │          │     │
│  │ Generation: │        [Ramesh]    [Suresh]   [Sita]   │
│  │ 1-5 (slider)│        Gen 2       Gen 2      Gen 2    │
│  │          │            │          │          │     │
│  │ [Reset]  │            │          │          │     │
│  └──────────┘        [Children]  [Children] [Children]  │
│                           │          │          │     │
│                       [Grandkids] [Grandkids] [Etc...]  │
│                                                         │
│  [Zoom: - + ]  [Fit to Screen]  [Export as PDF]         │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

Member Profile



Admin Inbox



7.3 Component Library (Design System)

Color Palette

```
Primary Colors:
- Primary: #2563eb (Blue 600) - Trust, heritage
- Primary Light: #3b82f6 (Blue 500)
- Primary Dark: #1e40af (Blue 800)

Secondary Colors:
- Accent: #f59e0b (Amber 500) - Warmth, family
- Success: #10b981 (Green 500)
```

```
- Warning: #f59e0b (Amber 500)
- Error: #ef4444 (Red 500)

Neutral Colors:
- Text Primary: #111827 (Gray 900)
- Text Secondary: #6b7280 (Gray 500)
- Background: #ffffff (White)
- Surface: #f9fafb (Gray 50)
- Border: #e5e7eb (Gray 200)
```

Typography

```
Font Family:
- Headings: 'Inter', sans-serif
- Body: 'Inter', sans-serif
- Code: 'JetBrains Mono', monospace

Font Sizes:
- h1: 36px / 2.25rem (Page titles)
- h2: 30px / 1.875rem (Section headers)
- h3: 24px / 1.5rem (Subsections)
- h4: 20px / 1.25rem (Card titles)
- body: 16px / 1rem (Default text)
- small: 14px / 0.875rem (Captions, labels)
- tiny: 12px / 0.75rem (Footnotes)

Font Weights:
- Regular: 400
- Medium: 500
- Semibold: 600
- Bold: 700
```

Spacing System

```
Based on 4px grid:

xs: 4px
sm: 8px
md: 16px
lg: 24px
xl: 32px
2xl: 48px
3xl: 64px
```

Components

Button

```
// Primary Button
<button className="bg-primary text-white px-4 py-2 rounded-lg hover:bg-primary-dark transition">
  Add Member
</button>

// Secondary Button
<button className="bg-white text-primary border-2 border-primary px-4 py-2 rounded-lg hover:bg-primary hover:text-white transition">
  Cancel
</button>

// Icon Button
<button className="p-2 rounded-full hover:bg-gray-100">
  <Icon name="edit" size={20} />
</button>
```

Card

```
<div className="bg-white rounded-xl shadow-md p-6 border border-gray-200">
  <h3 className="text-xl font-semibold mb-2">Card Title</h3>
  <p className="text-gray-600">Card content goes here</p>
</div>
```

Input Field

```
<div className="mb-4">
  <label className="block text-sm font-medium text-gray-700 mb-2">
    Full Name
  </label>
  <input
    type="text"
    className="w-full px-4 py-2 border border-gray-300 rounded-lg focus:ring-2 focus:ring-primary focus:border-transparent"
    placeholder="Enter full name"
  />
</div>
```

Tag

```
<span className="inline-block bg-blue-100 text-blue-800 text-sm px-3 py-1 rounded-full">
  Wedding
</span>
```

Avatar

```
<div className="relative">
  <img
    src="/path/to/photo.jpg"
    alt="Ramesh Dewangan"
    className="w-12 h-12 rounded-full object-cover border-2 border-white shadow"
  />
  {isLiving && (
    <span className="absolute bottom-0 right-0 w-3 h-3 bg-green-500 rounded-full border-2 border-white"></span>
  )}
</div>
```

7.4 Responsive Design

Breakpoints

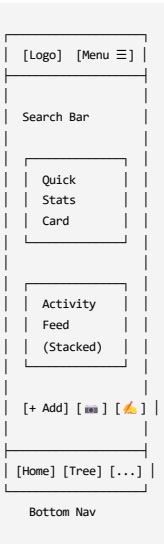
```
/* Mobile First Approach */

/* Small devices (phones, 0px and up) */
/* Default styles */

/* Medium devices (tablets, 768px and up) */
@media (min-width: 768px) {
  /* Adjust layout, show sidebar */
}

/* Large devices (desktops, 1024px and up) */
@media (min-width: 1024px) {
  /* Multi-column layouts, larger components */
}

/* Extra large devices (large desktops, 1280px and up) */
@media (min-width: 1280px) {
  /* Max content width, more whitespace */
}
```

Mobile Optimizations

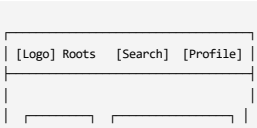
- Touch targets minimum 44x44px
- Bottom navigation on mobile (instead of top)
- Swipe gestures for navigation
- Collapsible filters (drawer on mobile)
- Infinite scroll instead of pagination
- Upload via camera integration
- Simplified forms (fewer fields per screen)

Responsive Layout Examples

Mobile (< 768px)



Tablet (768px - 1024px)



```
│ │ Stats   │ │   Activity      │ │
│ │ Card    │ │   Feed          │ │
│ └─────────┘ │                 │ │
│             │                 │ │
│ ┌─────────┐ │                 │ │
│ │ Quick   │ │                 │ │
│ │ Actions │ │                 │ │
│ └─────────┘ └─────────────────┘ │
│                                 │
└─────────────────────────────────┘
```

Desktop (> 1024px)

```
┌──────────────────────────────────────────────────┐
│ [Logo] Roots of Dewangan  [Search]    [Profile] [🔔] │
├──────────────────────────────────────────────────┤
│                                                  │
│ ┌─────────┐  ┌──────────────┐  ┌──────────┐  │
│ │         │  │              │  │          │  │
│ │  Stats  │  │   Activity   │  │ Birthdays │  │
│ │  Widget │  │   Feed       │  │          │  │
│ │         │  │              │  │          │  │
│ └─────────┘  │              │  └──────────┘  │
│              │              │                │
│ ┌─────────┐  │              │  ┌──────────┐  │
│ │  Quick  │  │              │  │ Reminders │  │
│ │ Actions │  │              │  │          │  │
│ └─────────┘  └──────────────┘  └──────────┘  │
│                                              │
└──────────────────────────────────────────────┘
   3-column layout with sidebar navigation
```

7.5 Navigation Patterns

Primary Navigation (Desktop)

```
┌──────────────────────────────────────────────┐
│ [Logo]  Home  Tree  Members  Media  Stories  Analytics │
└──────────────────────────────────────────────┘
```

Mobile Navigation (Bottom Sheet)

```
┌──────────────────────────────┐
│                              │
│       Main Content Area      │
│                              │
├──────────────────────────────┤
│ [🏠] [🌳] [👥] [📷] [•••]      │
│ Home Tree Members Media More  │
└──────────────────────────────┘
```

Sidebar Navigation (Admin Panel)

```
┌─────────────┬────────────────┐
│             │                │
│ Dashboard   │                │
│ Inbox (5)   │  Main Content  │
│ Members     │                │
│ Media       │                │
│ Analytics   │                │
│ Settings    │                │
│             │                │
└─────────────┴────────────────┘
```

7.6 Interaction Patterns

Loading States

```
// Skeleton Loader
<div className="animate-pulse">
  <div className="h-4 bg-gray-200 rounded w-3/4 mb-2"></div>
  <div className="h-4 bg-gray-200 rounded w-1/2"></div>
</div>

// Spinner
<div className="flex items-center justify-center">
  <div className="animate-spin rounded-full h-8 w-8 border-b-2 border-primary"></div>
</div>

// Progress Bar
<div className="w-full bg-gray-200 rounded-full h-2">
  <div className="bg-primary h-2 rounded-full" style={{width: '65%'}}></div>
</div>
```

Empty States

```
<div className="text-center py-12">
  <svg className="mx-auto h-12 w-12 text-gray-400">
    {/* Icon */}
  </svg>
  <h3 className="mt-2 text-sm font-medium text-gray-900">No stories yet</h3>
  <p className="mt-1 text-sm text-gray-500">
    Get started by sharing your first family story.
  </p>
  <button className="mt-6 btn-primary">
    Share a Story
  </button>
</div>
```

Error States

```
<div className="bg-red-50 border-l-4 border-red-400 p-4">
  <div className="flex">
    <div className="flex-shrink-0">
      <svg className="h-5 w-5 text-red-400">
        {/* Error icon */}
      </svg>
    </div>
    <div className="ml-3">
      <p className="text-sm text-red-700">
        Failed to upload photo. Please try again.
      </p>
    </div>
  </div>
</div>
```

Success States

```
<div className="bg-green-50 border-l-4 border-green-400 p-4">
  <div className="flex">
    <div className="flex-shrink-0">
      <svg className="h-5 w-5 text-green-400">
        {/* Success icon */}
      </svg>
    </div>
    <div className="ml-3">
      <p className="text-sm text-green-700">
        Member added successfully!
      </p>
    </div>
  </div>
</div>
```

Modal/Dialog

```
<div className="fixed inset-0 bg-black bg-opacity-50 flex items-center justify-center z-50">
  <div className="bg-white rounded-lg max-w-md w-full p-6">
    <div className="flex items-center justify-between mb-4">
      <h2 className="text-xl font-semibold">Delete Member</h2>
      <button className="text-gray-400 hover:text-gray-600">
        <svg className="h-6 w-6">×</svg>
      </button>
    </div>

    <p className="text-gray-600 mb-6">
      Are you sure you want to delete this member? This action cannot be undone.
    </p>

    <div className="flex gap-3 justify-end">
      <button className="btn-secondary">Cancel</button>
      <button className="btn-danger">Delete</button>
    </div>
  </div>
</div>
```

Toast Notifications

```
<div className="fixed bottom-4 right-4 bg-white shadow-lg rounded-lg p-4 flex items-center gap-3 z-50">
  <svg className="h-5 w-5 text-green-500">
    {/* Check icon */}
  </svg>
  <span className="text-sm font-medium">Photo uploaded successfully</span>
  <button className="text-gray-400 hover:text-gray-600">×</button>
</div>
```

7.7 Accessibility (a11y)

WCAG 2.1 AA Compliance

```
// Semantic HTML
<nav aria-label="Primary navigation">
  <ul role="list">
    <li><a href="/tree">Family Tree</a></li>
```

```
    <li><a href="/members">Members</a></li>
  </ul>
</nav>

// ARIA Labels
<button aria-label="Close dialog">×</button>
<input type="text" aria-label="Search members" placeholder="Search..." />

// Keyboard Navigation
<div
  role="button"
  tabIndex={0}
  onKeyDown={(e) => {
    if (e.key === 'Enter' || e.key === ' ') {
      handleClick();
    }
  }}
>
  Clickable div
</div>

// Focus Management
<button
  className="focus:ring-2 focus:ring-primary focus:outline-none"
  autoFocus
>
  Primary Action
</button>

// Color Contrast
// Text on background must have 4.5:1 ratio
// Large text (18pt+) must have 3:1 ratio

// Screen Reader Support
<img src="photo.jpg" alt="Ramesh Dewangan at his shop in 1985" />
<span className="sr-only">Loading...</span> {/* Screen reader only */}
```

Keyboard Shortcuts

```
Global:
- / : Focus search
- ? : Show keyboard shortcuts
- Esc : Close modal/dialog

Navigation:
- g h : Go to home
- g t : Go to tree
- g m : Go to members
- g a : Go to analytics

Actions:
- n m : New member
- n s : New story
- n p : Upload photo
```

7.8 Dark Mode Support (Future)

```
/* Tailwind Dark Mode */
.dark {
  --color-bg: #111827;
  --color-surface: #1f2937;
  --color-text: #f9fafb;
  --color-text-secondary: #9ca3af;
}

/* Component Example */
<div className="bg-white dark:bg-gray-800 text-gray-900 dark:text-gray-100">
  Content
</div>
```

7.9 Animation & Transitions

Micro-interactions

```
/* Hover Effects */
.card {
  transition: transform 0.2s, box-shadow 0.2s;
}
.card:hover {
  transform: translateY(-2px);
  box-shadow: 0 10px 20px rgba(0,0,0,0.1);
}

/* Button Press */
.button:active {
  transform: scale(0.98);
}

/* Loading Spinner */
@keyframes spin {
  to { transform: rotate(360deg); }
}
```

```
.spinner {
  animation: spin 1s linear infinite;
}

/* Fade In */
@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
.fade-in {
  animation: fadeIn 0.3s ease-in;
}

/* Slide Up */
@keyframes slideUp {
  from {
    transform: translateY(20px);
    opacity: 0;
  }
  to {
    transform: translateY(0);
    opacity: 1;
  }
}
.slide-up {
  animation: slideUp 0.3s ease-out;
}
```

Page Transitions

```
import { motion } from 'framer-motion';

<motion.div
  initial={{ opacity: 0, y: 20 }}
  animate={{ opacity: 1, y: 0 }}
  exit={{ opacity: 0, y: -20 }}
  transition={{ duration: 0.3 }}
>
  Page content
</motion.div>
```

7.10 Performance Optimization

Image Optimization

```
// Next.js Image Component
import Image from 'next/image';

<Image
  src="/photos/member.jpg"
  alt="Ramesh Dewangan"
  width={400}
  height={400}
  loading="lazy"
  placeholder="blur"
  blurDataURL="..."
/>
```

Code Splitting

```
// Dynamic imports
import dynamic from 'next/dynamic';

const FamilyTree = dynamic(() => import('@/components/FamilyTree'), {
  loading: () => <div>Loading tree...</div>,
  ssr: false // Client-side only
});
```

Virtual Scrolling

```
// For large lists (1000+ items)
import { FixedSizeList } from 'react-window';

<FixedSizeList
  height={600}
  itemCount={members.length}
  itemSize={80}
  width="100%"
>
  {({ index, style }) => (
    <div style={style}>
      <MemberCard member={members[index]} />
    </div>
  )}
</FixedSizeList>
```

Memoization

```
import { memo, useMemo } from 'react';

// Memoize expensive component
const MemberCard = memo(({ member }) => {
  return <div>{member.full_name}</div>;
});

// Memoize expensive calculation
const sortedMembers = useMemo(() => {
  return members.sort((a, b) =>
    a.full_name.localeCompare(b.full_name)
  );
}, [members]);
```

7.11 Form Design

Multi-step Form Pattern

```
// Add Member Form - Step 1
<form className="space-y-4">
  <h2 className="text-xl font-semibold">Basic Information</h2>

  <div>
    <label>Full Name *</label>
    <input type="text" required />
  </div>

  <div>
    <label>Nickname</label>
    <input type="text" />
  </div>

  <div className="grid grid-cols-2 gap-4">
    <div>
      <label>Gender</label>
      <select>
        <option>Male</option>
        <option>Female</option>
        <option>Other</option>
      </select>
    </div>
    <div>
      <label>Date of Birth</label>
      <input type="date" />
    </div>
  </div>

  {/* Progress indicator */}
  <div className="flex items-center justify-between pt-4">
    <span className="text-sm text-gray-500">Step 1 of 3</span>
    <div className="flex gap-2">
      <div className="w-20 h-2 bg-primary rounded"></div>
      <div className="w-20 h-2 bg-gray-200 rounded"></div>
      <div className="w-20 h-2 bg-gray-200 rounded"></div>
    </div>
    <button type="button" className="btn-primary">
      Next →
    </button>
  </div>
</form>
```

Inline Validation

```
<div className={error ? 'border-red-500' : 'border-gray-300'}>
  <input
    type="email"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
    onBlur={validateEmail}
  />
  {error && (
    <p className="text-red-500 text-sm mt-1">
      {error}
    </p>
  )}
  {!error && touched && (
    <p className="text-green-500 text-sm mt-1 flex items-center gap-1">
      <svg className="h-4 w-4">✓</svg>
      Valid email
    </p>
  )}
</div>
```

File Upload UI

```
<div className="border-2 border-dashed border-gray-300 rounded-lg p-8 text-center">
  <input
    type="file"
    id="file-upload"
    className="hidden"
    multiple
```

```
      accept="image/*"
      onChange={handleFileSelect}
    />
    <label
      htmlFor="file-upload"
      className="cursor-pointer"
    >
      <svg className="mx-auto h-12 w-12 text-gray-400">
        {/* Upload icon */}
      </svg>
      <p className="mt-2 text-sm text-gray-600">
        <span className="font-medium text-primary">Click to upload</span>
        {' '}or drag and drop
      </p>
      <p className="text-xs text-gray-500">
        PNG, JPG, GIF up to 10MB
      </p>
    </label>

    {/* Selected files */}
    {files.length > 0 && (
      <div className="mt-4 space-y-2">
        {files.map(file => (
          <div key={file.name} className="flex items-center justify-between bg-gray-50 p-2 rounded">
            <span className="text-sm truncate">{file.name}</span>
            <button onClick={() => removeFile(file)}>×</button>
          </div>
        ))}
      </div>
    )}
  </div>
```

8. API SPECIFICATIONS

8.1 API Architecture

REST API Endpoints

Base URL: `https://roots-of-dewangan.vercel.app/api`

All endpoints return JSON unless otherwise specified.

Authentication: Bearer token in `Authorization` header (Supabase JWT)

Standard Response Format

```
{
  "success": true | false,
  "data": { ... } | null,
  "error": {
    "code": "ERROR_CODE",
    "message": "Human-readable error message"
  } | null,
  "meta": {
    "timestamp": "2026-02-05T10:30:00Z",
    "requestId": "uuid"
  }
}
```

8.2 Endpoint Specifications

Family Members

GET /api/members

Retrieve list of family members

Query Parameters:

- `lineage` (string): 'Father' | 'Mother' | 'Both'
- `status` (string): 'Living' | 'Deceased'
- `generation` (number): Filter by generation
- `limit` (number): Max results (default: 50, max: 100)
- `offset` (number): Pagination offset
- `search` (string): Search query

Example Request:

```
GET /api/members?lineage=Father&status=Living&limit=20
Authorization: Bearer eyJhbGc...
```

Response:

```
{
  "success": true,
  "data": {
    "members": [
      {
        "id": "uuid",
```

```
        "full_name": "Ramesh Kumar Dewangan",
        "nickname": "Ramu",
        "gender": "Male",
        "date_of_birth": "1952-03-15",
        "date_of_death": "2018-11-22",
        "status": "Deceased",
        "lineage": "Father",
        "generation": 3,
        "profile_photo_url": "https://...",
        "age": 66,
        "parent1_id": "uuid",
        "parent2_id": "uuid",
        "children_count": 3,
        "media_count": 23,
        "story_count": 5
      }
    ],
    "total": 142,
    "limit": 20,
    "offset": 0
  }
}
```

GET /api/members/:id

Get single member with full details

Example Request:

```
GET /api/members/550e8400-e29b-41d4-a716-446655440000
Authorization: Bearer eyJhbGc...
```

Response:

```
{
  "success": true,
  "data": {
    "member": {
      "id": "uuid",
      "full_name": "Ramesh Kumar Dewangan",
      "nickname": "Ramu",
      "gender": "Male",
      "date_of_birth": "1952-03-15",
      "date_of_death": "2018-11-22",
      "birth_place": "Raipur, Chhattisgarh, India",
      "current_location": null,
      "status": "Deceased",
      "lineage": "Father",
      "generation": 3,
      "bio_summary": "Pioneering businessman...",
      "profile_photo_url": "https://...",

      "parents": [
        {
          "id": "uuid",
          "full_name": "Gopal Dewangan",
          "relationship": "Father"
        },
        {
          "id": "uuid",
          "full_name": "Sita Devi",
          "relationship": "Mother"
        }
      ],

      "spouses": [
        {
          "id": "uuid",
          "full_name": "Meena Dewangan",
          "marriage_date": "1975-01-01",
          "status": "Married"
        }
      ],

      "children": [
        {
          "id": "uuid",
          "full_name": "Rahul Dewangan"
        },
        {
          "id": "uuid",
          "full_name": "Priya Sharma"
        },
        {
          "id": "uuid",
          "full_name": "Somu Dewangan"
        }
      ],

      "occupations": [
        {
          "occupation_name": "Farmer",
          "start_year": 1970,
          "end_year": 1980
        },
```

```
        {
          "occupation_name": "Small Business Owner",
          "start_year": 1980,
          "end_year": 2015
        }
      ],

      "education": [
        {
          "degree": "8th Standard",
          "institution_name": "Village School",
          "year_completed": 1965
        }
      ],

      "media_count": 23,
      "story_count": 5,
      "event_count": 12
    }
  }
}
```

POST /api/members

Create new family member

Request Headers:

```
Content-Type: application/json
Authorization: Bearer eyJhbGc...
```

Request Body:

```
{
  "full_name": "Rajesh Dewangan",
  "nickname": "Raju",
  "gender": "Male",
  "date_of_birth": "1985-06-12",
  "birth_place": "Mumbai, India",
  "lineage": "Father",
  "status": "Living",
  "parent1_id": "uuid-of-father",
  "parent2_id": "uuid-of-mother",
  "bio_summary": "Software engineer..."
}
```

Response:

```
{
  "success": true,
  "data": {
    "member": {
      "id": "newly-generated-uuid",
      "full_name": "Rajesh Dewangan",
      "generation": 4,
      "created_at": "2026-02-05T10:30:00Z"
    },
    "message": "Member created successfully"
  }
}
```

Error Response (400):

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid date of birth: cannot be in the future",
    "field": "date_of_birth"
  }
}
```

PUT /api/members/:id

Update existing member

Request Body: (all fields optional)

```
{
  "bio_summary": "Updated biography text...",
  "current_location": "Delhi, India"
}
```

Response:

```
{
  "success": true,
```

```
  "data": {
    "member": {
      "id": "uuid",
      "full_name": "Rajesh Dewangan",
      "bio_summary": "Updated biography text...",
      "current_location": "Delhi, India",
      "last_modified": "2026-02-05T11:15:00Z"
    },
    "message": "Member updated successfully"
  }
}
```

DELETE /api/members/:id

Soft delete member (requires Super Admin)

Request:

```
DELETE /api/members/550e8400-e29b-41d4-a716-446655440000
Authorization: Bearer eyJhbGc... (Super Admin token)
```

Response:

```
{
  "success": true,
  "data": {
    "message": "Member deleted successfully",
    "deleted_id": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

Media

GET /api/media

Retrieve media files

Query Parameters:

- `type` (string): 'Photo' | 'Video' | 'Audio' | 'Document'
- `member_id` (uuid): Filter by linked member
- `event_id` (uuid): Filter by event
- `tags` (string[]): Filter by tags (comma-separated)
- `date_from` (date): Min date (YYYY-MM-DD)
- `date_to` (date): Max date (YYYY-MM-DD)
- `limit` (number): Max results (default: 50)
- `offset` (number): Pagination offset

Example Request:

```
GET /api/media?type=Photo&member_id=uuid&limit=10
Authorization: Bearer eyJhbGc...
```

Response:

```
{
  "success": true,
  "data": {
    "media": [
      {
        "id": "uuid",
        "filename": "wedding-2024.webp",
        "file_type": "Photo",
        "r2_url": "https://pub-xxx.r2.dev/photos/wedding-2024.webp",
        "thumbnail_url": "https://pub-xxx.r2.dev/thumbs/wedding-2024.webp",
        "date_taken": "2024-11-15",
        "location": "Raipur, India",
        "description": "Wedding ceremony",
        "tags": ["wedding", "family", "celebration"],
        "ai_description": "Group photo at wedding ceremony in garden",
        "linked_members": [
          {"id": "uuid", "full_name": "Somu Dewangan"},
          {"id": "uuid", "full_name": "Priya Sharma"}
        ],
        "uploaded_by": "admin@example.com",
        "upload_date": "2024-11-20T10:30:00Z",
        "original_size_mb": 5.2,
        "compressed_size_mb": 1.5,
        "compression_ratio": 0.29
      }
    ],
    "total": 856,
    "limit": 10,
    "offset": 0
  }
}
```

POST /api/upload

Upload media file

Request:

- Content-Type: `multipart/form-data`
- Authorization: Bearer token

Form Data:

- `file`: File object (required)
- `file_type`: 'Photo' | 'Video' | 'Audio' | 'Document' (required)
- `description` (optional): Text description
- `date_taken` (optional): YYYY-MM-DD
- `location` (optional): String
- `member_ids` (optional): Array of UUIDs (JSON string)
- `tags` (optional): Array of strings (JSON string)

Example Request (using fetch):

```
const formData = new FormData();
formData.append('file', fileObject);
formData.append('file_type', 'Photo');
formData.append('description', 'Family gathering');
formData.append('date_taken', '2024-12-25');
formData.append('member_ids', JSON.stringify(['uuid1', 'uuid2']));
formData.append('tags', JSON.stringify(['christmas', 'family']));

const response = await fetch('/api/upload', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${token}`
  },
  body: formData
});
```

Response:

```
{
  "success": true,
  "data": {
    "media": {
      "id": "uuid",
      "filename": "family-gathering.webp",
      "r2_url": "https://pub-xxx.r2.dev/photos/2024/12/family-gathering.webp",
      "thumbnail_url": "https://pub-xxx.r2.dev/thumbs/2024/12/family-gathering.webp"
    },
    "stats": {
      "original_size_mb": "5.20",
      "compressed_size_mb": "1.50",
      "compression_ratio": "29%",
      "saved_mb": "3.70"
    }
  }
}
```

Error Response (413 - File Too Large):

```
{
  "success": false,
  "error": {
    "code": "FILE_TOO_LARGE",
    "message": "File size exceeds maximum allowed (50MB for photos)",
    "max_size_mb": 50,
    "file_size_mb": 75.3
  }
}
```

Stories

GET /api/stories

Retrieve stories

Query Parameters:

- `type` (string): Story type
- `member_id` (uuid): Filter by member
- `theme` (string): Filter by theme
- `search` (string): Full-text search
- `limit` (number): Max results (default: 50)
- `offset` (number): Pagination offset

Example Request:

```
GET /api/stories?type=Achievement&limit=5
Authorization: Bearer eyJhbGc...
```

Response:

```
{
  "success": true,
  "data": {
    "stories": [
      {
        "id": "uuid",
        "title": "The Flood of 1975",
        "story_text": "During the monsoon of 1975...",
        "story_type": "Achievement",
        "storyteller": "Meena Dewangan",
        "event_date": "1975-08-20",
        "location": "Raipur",
        "themes": ["resilience", "community_service"],
        "ai_summary": "Ramesh organized rescue during 1975 flood, saving elderly neighbors.",
        "ai_sentiment": "Positive",
        "linked_members": [
          {"id": "uuid", "full_name": "Ramesh Dewangan"}
        ],
        "audio_url": null,
        "transcribed": false,
        "added_by": "meena@example.com",
        "added_date": "2024-12-01T15:20:00Z"
      }
    ],
    "total": 47,
    "limit": 5,
    "offset": 0
  }
}
```

POST /api/stories

Create new story

Request Body:

```
{
  "title": "My Grandfather's Journey",
  "story_text": "Full story text here... (minimum 50 characters)",
  "story_type": "Migration",
  "event_date": "1960-05-01",
  "location": "Raipur to Mumbai",
  "themes": ["migration", "resilience"],
  "member_ids": ["uuid1", "uuid2"],
  "audio_url": "https://..." // optional
}
```

Response:

```
{
  "success": true,
  "data": {
    "story": {
      "id": "newly-generated-uuid",
      "title": "My Grandfather's Journey",
      "ai_processed": false,
      "created_at": "2026-02-05T10:30:00Z"
    },
    "message": "Story created successfully. AI processing queued."
  }
}
```

Events

GET /api/events

Retrieve events

Query Parameters:

- `type` (string): Event type
- `year` (number): Filter by year
- `member_id` (uuid): Events attended by member
- `limit` (number): Max results (default: 50)
- `offset` (number): Pagination offset

Example Request:

```
GET /api/events?type=Wedding&year=2024
Authorization: Bearer eyJhbGc...
```

Response:

```
{
  "success": true,
  "data": {
```

```
    "events": [
      {
        "id": "uuid",
        "name": "Somu's Wedding",
        "event_type": "Wedding",
        "event_date": "2024-11-15",
        "location": "Raipur, India",
        "description": "Traditional wedding ceremony...",
        "cultural_significance": "First wedding in family with 500+ guests",
        "attendee_count": 35,
        "media_count": 120,
        "story_count": 3,
        "added_by": "admin@example.com",
        "added_date": "2024-11-20T10:00:00Z"
      }
    ],
    "total": 34,
    "limit": 50,
    "offset": 0
  }
}
```

POST /api/events

Create new event

Request Body:

```
{
  "name": "Family Reunion 2026",
  "event_type": "Reunion",
  "event_date": "2026-07-15",
  "location": "Raipur, India",
  "description": "Annual family gathering at ancestral home",
  "cultural_significance": "Continuing 50-year tradition",
  "attendee_ids": ["uuid1", "uuid2", "uuid3"]
}
```

Response:

```
{
  "success": true,
  "data": {
    "event": {
      "id": "newly-generated-uuid",
      "name": "Family Reunion 2026",
      "event_date": "2026-07-15",
      "attendee_count": 3
    },
    "message": "Event created successfully"
  }
}
```

Search

GET /api/search

Universal search across all entities

Query Parameters:

- `q` (required): Search query (string)
- `types` (array): ['members', 'stories', 'media', 'events'] (default: all)
- `limit` (number): Max results per type (default: 5)

Example Request:

```
GET /api/search?q=ramesh&types=members,stories&limit=3
Authorization: Bearer eyJhbGc...
```

Response:

```
{
  "success": true,
  "data": {
    "members": [
      {
        "id": "uuid",
        "full_name": "Ramesh Kumar Dewangan",
        "profile_photo_url": "https://...",
        "relevance_score": 0.95
      }
    ],
    "stories": [
      {
        "id": "uuid",
        "title": "The Flood of 1975",
        "excerpt": "...Ramesh organized rescue...",
        "relevance_score": 0.87
      }
```

```
    ],
    "media": [],
    "events": [],
    "total_results": 2
  }
}
```

POST /api/search/semantic

Semantic search using AI embeddings

Request Body:

```
{
  "query": "stories about overcoming challenges",
  "types": ["stories"],
  "limit": 10,
  "threshold": 0.7
}
```

Response:

```
{
  "success": true,
  "data": {
    "stories": [
      {
        "id": "uuid",
        "title": "The Flood of 1975",
        "story_text": "During the monsoon of 1975...",
        "similarity_score": 0.89
      },
      {
        "id": "uuid",
        "title": "Starting the Business",
        "story_text": "With only 100 rupees...",
        "similarity_score": 0.82
      }
    ],
    "total": 2
  }
}
```

Admin Endpoints

GET /api/admin/inbox

Get submission queue

Query Parameters:

- `status` (string): 'Pending' | 'Approved' | 'Rejected' | 'Needs Info' | 'Duplicate'
- `type` (string): Submission type
- `limit` (number): Max results (default: 50)
- `offset` (number): Pagination offset

Example Request:

```
GET /api/admin/inbox?status=Pending&limit=10
Authorization: Bearer eyJhbGc... (Admin token)
```

Response:

```
{
  "success": true,
  "data": {
    "submissions": [
      {
        "id": "uuid",
        "submission_type": "New Member",
        "status": "Pending",
        "raw_data": {
          "full_name": "Rajesh Dewangan",
          "nickname": "Raju",
          "date_of_birth": "1985-06-12",
          "parent1_name": "Ramesh Dewangan",
          "parent2_name": "Meena Dewangan"
        },
        "submitter_name": "Priya Sharma",
        "submitter_email": "priya@example.com",
        "submission_date": "2026-02-05T14:30:00Z",
        "possible_duplicate": {
          "id": "existing-uuid",
          "full_name": "Rajesh Kumar Dewangan",
          "date_of_birth": "1985-06-15",
          "similarity_score": 0.85
        }
      }
    ],
    "counts": {
```

```
      "pending": 5,
      "approved": 23,
      "rejected": 2,
      "needs_info": 1,
      "duplicate": 0
    },
    "total": 5,
    "limit": 10,
    "offset": 0
  }
}
```

POST /api/admin/inbox/:id/approve

Approve submission and create record

Request Body:

```
{
  "linked_record_id": "uuid", // Optional: if linking to existing member
  "notes": "Approved - verified with family",
  "modifications": { // Optional: modify data before approval
    "full_name": "Rajesh Kumar Dewangan"
  }
}
```

Response:

```
{
  "success": true,
  "data": {
    "inbox_submission": {
      "id": "submission-uuid",
      "status": "Approved",
      "reviewed_by": "admin@example.com",
      "review_date": "2026-02-05T15:00:00Z"
    },
    "created_record": {
      "id": "newly-created-uuid",
      "type": "family_member",
      "full_name": "Rajesh Kumar Dewangan"
    },
    "message": "Submission approved and member created successfully"
  }
}
```

POST /api/admin/inbox/:id/reject

Reject submission with reason

Request Body:

```
{
  "reason": "Duplicate of existing member (Rajesh Kumar Dewangan)"
}
```

Response:

```
{
  "success": true,
  "data": {
    "inbox_submission": {
      "id": "submission-uuid",
      "status": "Rejected",
      "reviewed_by": "admin@example.com",
      "review_date": "2026-02-05T15:00:00Z",
      "review_notes": "Duplicate of existing member"
    },
    "message": "Submission rejected. Email sent to submitter."
  }
}
```

POST /api/admin/inbox/:id/request-info

Request more information from submitter

Request Body:

```
{
  "message": "Please provide parent's full names and dates of birth for verification."
}
```

Response:

```
{
  "success": true,
```

```
  "data": {
    "inbox_submission": {
      "id": "submission-uuid",
      "status": "Needs Info",
      "review_notes": "Awaiting additional parent information"
    },
    "message": "Information request sent to submitter"
  }
}
```

GET /api/admin/statistics

Get admin dashboard statistics

Example Request:

```
GET /api/admin/statistics
Authorization: Bearer eyJhbGc... (Admin token)
```

Response:

```
{
  "success": true,
  "data": {
    "overview": {
      "total_members": 142,
      "total_media": 856,
      "total_stories": 47,
      "total_events": 34
    },
    "inbox": {
      "pending": 5,
      "needs_info": 1,
      "approved_this_week": 12,
      "rejected_this_week": 1
    },
    "recent_activity": {
      "last_submission": "2026-02-05T14:30:00Z",
      "last_approval": "2026-02-05T15:00:00Z",
      "active_contributors": 8
    },
    "data_quality": {
      "profiles_complete": 85.2,
      "profiles_with_photos": 67.3,
      "profiles_with_bio": 42.1
    },
    "storage": {
      "database_size_mb": 45.3,
      "media_storage_gb": 2.8,
      "backup_last_run": "2026-02-05T02:00:00Z"
    }
  }
}
```

AI Operations

POST /api/ai/tag-image

AI-tag uploaded image (face detection + object recognition)

Request Body:

```
{
  "media_id": "uuid"
}
```

Response:

```
{
  "success": true,
  "data": {
    "detected_faces": [
      {
        "coordinates": {
          "x": 100,
          "y": 150,
          "width": 50,
          "height": 50
        },
        "suggested_member_id": "uuid-of-ramesh",
        "confidence": 0.92
      },
      {
        "coordinates": {
          "x": 250,
          "y": 160,
          "width": 48,
          "height": 48
        },
        "suggested_member_id": null,
```

```
        "confidence": 0.75
      }
    ],
    "ai_description": "Wedding ceremony in garden with family members",
    "ai_objects": ["wedding", "garden", "celebration", "people", "flowers"],
    "processed": true
  }
}
```

POST /api/ai/transcribe-audio

Transcribe audio file using Whisper

Request Body:

```
{
  "audio_url": "https://pub-xxx.r2.dev/audio/story-recording.mp3"
}
```

Response:

```
{
  "success": true,
  "data": {
    "transcript": "During the monsoon of 1975, the river overflowed its banks and flooded the entire village. My father, Ramesh, organized a rescue team...",
    "language": "en",
    "duration_seconds": 180,
    "confidence": 0.94
  }
}
```

Error Response (503 - AI Service Unavailable):

```
{
  "success": false,
  "error": {
    "code": "AI_ERROR",
    "message": "Cloudflare AI service temporarily unavailable",
    "retry_after": 60
  }
}
```

POST /api/ai/extract-themes

Extract themes from story text using LLaMA 2

Request Body:

```
{
  "text": "During the monsoon of 1975, the river overfloded and threatened the entire village. My father organized neighbors to evacuate elderly residents. They worked through the night without rest, saving dozens
}
```

Response:

```
{
  "success": true,
  "data": {
    "themes": ["resilience", "community_service", "family_unity", "sacrifice"],
    "summary": "Ramesh organized a rescue during the 1975 flood, saving elderly neighbors. His selfless act strengthened community bonds.",
    "sentiment": "Positive",
    "key_values": ["courage", "compassion", "leadership"]
  }
}
```

POST /api/ai/suggest-members

Suggest family member matches for untagged faces

Request Body:

```
{
  "media_id": "uuid",
  "face_index": 0
}
```

Response:

```
{
  "success": true,
  "data": {
    "suggestions": [
      {
        "member_id": "uuid-ramesh",
        "full_name": "Ramesh Kumar Dewangan",
```

```
        "confidence": 0.92,
        "reasoning": "Face match with 3 existing tagged photos"
      },
      {
        "member_id": "uuid-suresh",
        "full_name": "Suresh Dewangan",
        "confidence": 0.67,
        "reasoning": "Similar facial features, same generation"
      }
    ]
  }
}
```

Analytics

GET /api/analytics/statistics

Get aggregated family statistics

Example Request:

```
GET /api/analytics/statistics
Authorization: Bearer eyJhbGc...
```

Response:

```
{
  "success": true,
  "data": {
    "demographics": {
      "total_members": 142,
      "living_members": 89,
      "deceased_members": 53,
      "male_count": 78,
      "female_count": 64,
      "gender_ratio": 1.22
    },
    "generations": {
      "max_generation": 5,
      "members_by_generation": {
        "1": 2,
        "2": 12,
        "3": 35,
        "4": 58,
        "5": 35
      }
    },
    "life_statistics": {
      "avg_lifespan": 68.5,
      "oldest_living": 89,
      "youngest_member": 2
    },
    "content": {
      "total_media": 856,
      "total_stories": 47,
      "total_events": 34,
      "total_traditions": 12
    },
    "education": {
      "college_educated_count": 42,
      "college_educated_percentage": 29.6,
      "advanced_degrees": 18
    },
    "geography": {
      "countries": 3,
      "cities": 15,
      "most_common_location": "Raipur, India"
    }
  }
}
```

GET /api/analytics/occupation-trends

Get occupation trends across generations

Example Request:

```
GET /api/analytics/occupation-trends
Authorization: Bearer eyJhbGc...
```

Response:

```
{
  "success": true,
  "data": {
    "trends": [
      {
        "occupation_name": "Farmer",
        "category": "Agriculture",
        "member_count": 35,
```

```
        "first_year": 1920,
        "last_year": 2010,
        "generations_practiced": [1, 2, 3],
        "trend": "declining"
      },
      {
        "occupation_name": "Software Engineer",
        "category": "Technology",
        "member_count": 12,
        "first_year": 2005,
        "last_year": 2025,
        "generations_practiced": [4, 5],
        "trend": "rising"
      },
      {
        "occupation_name": "Teacher",
        "category": "Education",
        "member_count": 18,
        "first_year": 1950,
        "last_year": 2024,
        "generations_practiced": [2, 3, 4, 5],
        "trend": "stable"
      }
    ],
    "category_distribution": {
      "Agriculture": 35,
      "Technology": 12,
      "Education": 18,
      "Business": 22,
      "Healthcare": 8,
      "Other": 15
    }
  }
}
```

GET /api/analytics/migration-map

Get migration data for geographic visualization

Example Request:

```
GET /api/analytics/migration-map
Authorization: Bearer eyJhbGc...
```

Response:


```
{
  "success": true,
  "data": {
    "locations": [
      {
        "id": "uuid",
        "name": "Raipur, Chhattisgarh",
        "latitude": 21.2514,
        "longitude": 81.6296,
        "member_count": 45,
        "time_period": "1920-1980",
        "significance": "Ancestral village"
      },
      {
        "id": "uuid",
        "name": "Mumbai, Maharashtra",
        "latitude": 19.0760,
        "longitude": 72.8777,
        "member_count": 32,
        "time_period": "1960-2025",
        "significance": "Economic migration hub"
      },
      {
        "id": "uuid",
        "name": "Bangalore, Karnataka",
        "latitude": 12.9716,
        "longitude": 77.5946,
        "member_count": 18,
        "time_period": "2000-2025",
        "significance": "Tech industry migration"
      }
    ],
    "migrations": [
      {
        "from": {
          "name": "Raipur",
          "coords": [21.2514, 81.6296]
        },
        "to": {
          "name": "Mumbai",
          "coords": [19.0760, 72.8777]
        },
        "member_count": 15,
        "time_period": "1960-1980",
        "reason": "Economic opportunities"
      },
      {
        "from": {
          "name": "Mumbai",
```

```
          "coords": [19.0760, 72.8777]
        },
        "to": {
          "name": "Bangalore",
          "coords": [12.9716, 77.5946]
        },
        "member_count": 12,
        "time_period": "2000-2020",
        "reason": "Technology sector"
      }
    ]
  }
}
```

GET /api/analytics/timeline

Get chronological timeline of all family events

Query Parameters:

- `start_year` (number): Filter from year (optional)
- `end_year` (number): Filter to year (optional)
- `event_types` (array): Filter by event types (optional)

Example Request:

```
GET /api/analytics/timeline?start_year=1950&end_year=2000
Authorization: Bearer eyJhbGc...
```

Response:

```
{
  "success": true,
  "data": {
    "timeline": [
      {
        "year": 1952,
        "events": [
          {
            "type": "Birth",
            "date": "1952-03-15",
            "title": "Ramesh Dewangan born",
            "member_id": "uuid",
            "location": "Raipur"
          }
        ]
      },
      {
        "year": 1975,
        "events": [
          {
            "type": "Marriage",
            "date": "1975-01-01",
            "title": "Ramesh & Meena married",
            "member_ids": ["uuid1", "uuid2"],
            "location": "Raipur"
          },
          {
            "type": "Achievement",
            "date": "1975-08-20",
            "title": "Flood rescue organized",
            "story_id": "uuid",
            "location": "Raipur"
          }
        ]
      }
    ],
    "summary": {
      "total_events": 156,
      "births": 42,
      "deaths": 15,
      "marriages": 28,
      "achievements": 23,
      "other": 48
    }
  }
}
```

Export Endpoints

GET /api/export/gedcom

Export family tree as GEDCOM file

Example Request:

```
GET /api/export/gedcom
Authorization: Bearer eyJhbGc... (Admin token)
```

Response:

- Content-Type: `application/x-gedcom`
- Content-Disposition: `attachment; filename="dewangan-family-tree.ged"`
- Body: GEDCOM format file

GEDCOM Sample Output:

```
0 HEAD
1 SOUR Roots of Dewangan
1 DEST ANY
1 DATE 5 FEB 2026
1 CHAR UTF-8
1 GEDC
2 VERS 5.5.1
0 @I1@ INDI
1 NAME Ramesh Kumar /Dewangan/
1 SEX M
1 BIRT
2 DATE 15 MAR 1952
2 PLAC Raipur, Chhattisgarh, India
1 DEAT
2 DATE 22 NOV 2018
1 FAMC @F1@
1 FAMS @F2@
0 @F1@ FAM
1 HUSB @I0@
1 WIFE @I00@
1 CHIL @I1@
0 TRLR
```

GET /api/export/pdf

Generate PDF report of family data

Query Parameters:

- `member_ids` (array): Include specific members (optional, default: all)
- `include_photos` (boolean): Include media (default: true)
- `include_stories` (boolean): Include narratives (default: true)
- `template` (string): 'standard' | 'compact' | 'detailed' (default: standard)

Example Request:

```
GET /api/export/pdf?include_photos=true&template=detailed
Authorization: Bearer eyJhbGc... (Admin token)
```

Response:

- Content-Type: `application/pdf`
- Content-Disposition: `attachment; filename="dewangan-family-report-2026-02-05.pdf"`
- Body: PDF file

GET /api/export/full

Export complete database + media as ZIP

Example Request:

```
GET /api/export/full
Authorization: Bearer eyJhbGc... (Super Admin token only)
```

Response:

- Content-Type: `application/zip`
- Content-Disposition: `attachment; filename="roots-of-dewangan-full-export-2026-02-05.zip"`
- Body: ZIP file containing:
  - `database/family_members.csv`
  - `database/stories.csv`
  - `database/media.csv`
  - `database/events.csv`
  - `database/schema.sql`
  - `media/photos/...` (all image files)
  - `media/videos/...` (all video files)
  - `media/audio/...` (all audio files)
  - `README.md` (documentation)

8.3 Error Codes Reference

Code	HTTP Status	Description	Example
`AUTH_REQUIRED`	401	No authentication token provided	Missing Authorization header
`AUTH_INVALID`	401	Invalid or expired token	Malformed JWT
`AUTH_EXPIRED`	401	Token has expired	Token older than 30 days
`FORBIDDEN`	403	Insufficient permissions	Viewer trying to delete member
`NOT_FOUND`	404	Resource not found	Member ID doesn't exist

Code	HTTP Status	Description	Example
VALIDATION_ERROR	400	Invalid input data	DOB in future
DUPLICATE_ENTRY	409	Record already exists	Email already registered
FILE_TOO_LARGE	413	File exceeds size limit	Photo > 50MB
UNSUPPORTED_TYPE	415	Unsupported file type	.exe file upload
RATE_LIMIT	429	Too many requests	>120 req/min
SERVER_ERROR	500	Internal server error	Database connection failed
AI_ERROR	503	AI service unavailable	Cloudflare AI down
STORAGE_ERROR	503	Storage service unavailable	R2 temporarily down

8.4 Rate Limiting

```
Unauthenticated: 30 requests/minute
Authenticated: 120 requests/minute
Admin: 300 requests/minute
Super Admin: Unlimited

File uploads: 10/hour/user
AI operations: 100/day/user (free tier limit)
Export operations: 5/day/user
```

Rate Limit Headers:

```
X-RateLimit-Limit: 120
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1675612800
```

Rate Limit Exceeded Response:

```
{
  "success": false,
  "error": {
    "code": "RATE_LIMIT",
    "message": "Rate limit exceeded. Please try again in 45 seconds.",
    "retry_after": 45,
    "limit": 120,
    "window": "1 minute"
  }
}
```

8.5 Pagination

All list endpoints support pagination:

Request:

```
GET /api/members?limit=20&offset=40
```

Response includes pagination metadata:

```
{
  "success": true,
  "data": {
    "members": [...],
    "pagination": {
      "total": 142,
      "limit": 20,
      "offset": 40,
      "current_page": 3,
      "total_pages": 8,
      "has_next": true,
      "has_prev": true,
      "next_url": "/api/members?limit=20&offset=60",
      "prev_url": "/api/members?limit=20&offset=20"
    }
  }
}
```

8.6 Webhooks (Future Feature)

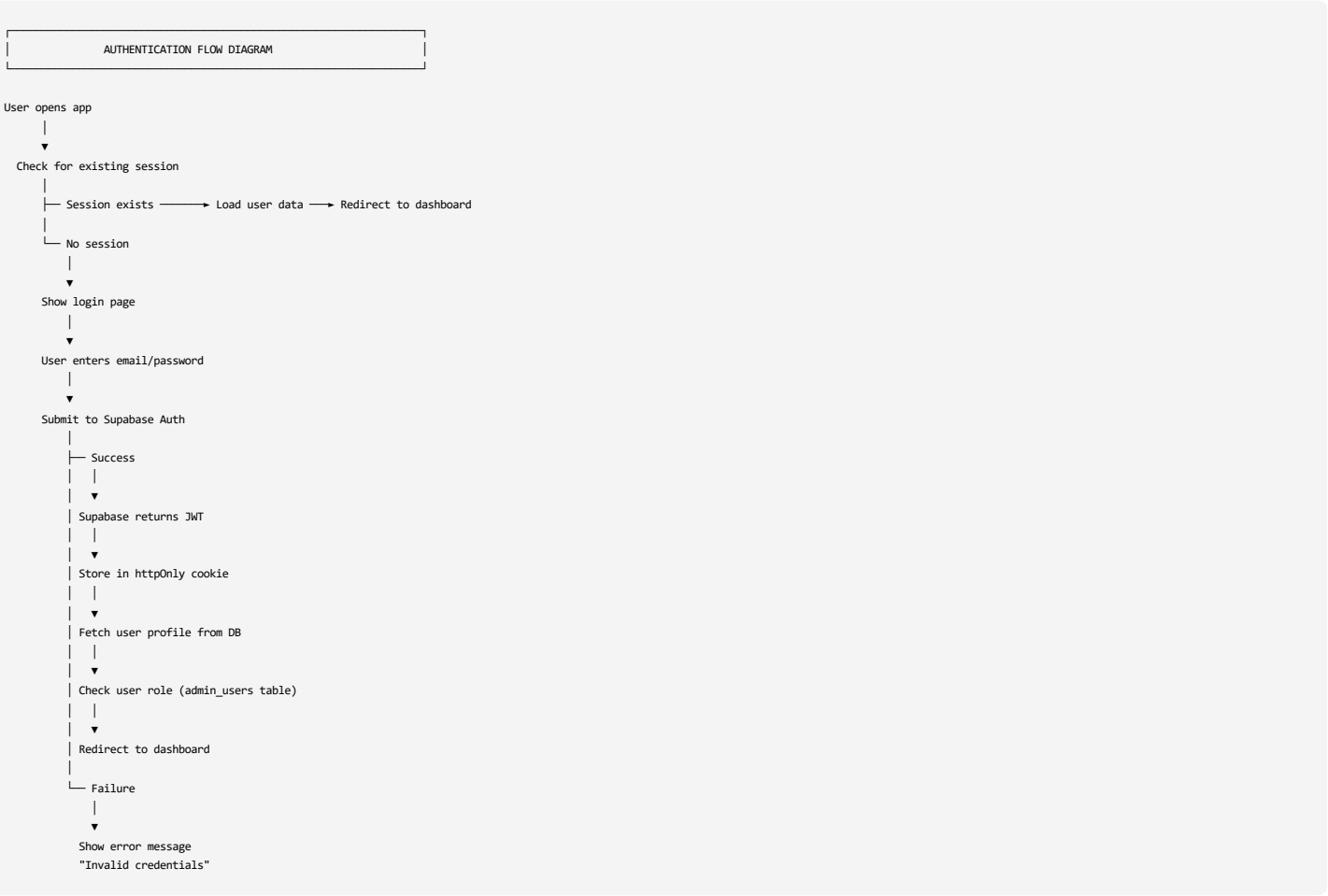
```
// Webhook payload structure (for future integration)
{
  "event": "member.created",
  "timestamp": "2026-02-05T10:30:00Z",
  "data": {
    "id": "uuid",
    "full_name": "Rajesh Dewangan",
    "created_by": "admin@example.com"
  },
  "signature": "sha256_hash_of_payload"
}
```

Supported events (future):

- `member.created`
- `member.updated`
- `member.deleted`
- `media.uploaded`
- `story.created`
- `submission.pending`
- `submission.approved`

9. SECURITY & PRIVACY

9.1 Authentication Flow



9.2 Authorization Matrix

Resource	Super Admin	Admin	Contributor	Viewer
Members				
View	✅	✅	✅	✅
Create	✅	✅	✅ (via form)	❌
Edit	✅	✅	❌	❌
Delete	✅	❌	❌	❌
View Contact Info	✅	✅	❌	❌
Media				
View	✅	✅	✅	✅
Upload	✅	✅	✅	❌
Edit Metadata	✅	✅	Own only	❌
Delete	✅	✅	Own only	❌
Stories				
View	✅	✅	✅	✅
Create	✅	✅	✅	❌
Edit	✅	✅	Own only	❌
Delete	✅	✅	Own only	❌
Inbox				
View	✅	✅	Own submissions	❌
Approve/Reject	✅	✅	❌	❌
Admin				
Manage Users	✅	❌	❌	❌

Resource	Super Admin	Admin	Contributor	Viewer
Export Data	✅	✅	❌	❌
System Settings	✅	❌	❌	❌
Backups	✅	View only	❌	❌

9.3 Data Encryption

At Rest

- **Database**: AES-256 encryption (Supabase default)
- **Files**: Encrypted storage in Cloudflare R2
- **Sensitive Fields**: Additional encryption for contact_info (JSONB field)

In Transit

- **All traffic**: TLS 1.3
- **API calls**: HTTPS only
- **Webhooks**: Signed with secret key

Sensitive Data Handling

```
// Encrypt sensitive data before storing
import { createCipheriv, createDecipheriv, randomBytes } from 'crypto';

const ENCRYPTION_KEY = process.env.ENCRYPTION_KEY; // 32-byte key
const ALGORITHM = 'aes-256-gcm';

function encryptContactInfo(data: object): string {
  const iv = randomBytes(16);
  const cipher = createCipheriv(ALGORITHM, Buffer.from(ENCRYPTION_KEY, 'hex'), iv);

  let encrypted = cipher.update(JSON.stringify(data), 'utf8', 'hex');
  encrypted += cipher.final('hex');

  const authTag = cipher.getAuthTag();

  return JSON.stringify({
    iv: iv.toString('hex'),
    encrypted: encrypted,
    authTag: authTag.toString('hex')
  });
}

function decryptContactInfo(encryptedData: string): object {
  const { iv, encrypted, authTag } = JSON.parse(encryptedData);

  const decipher = createDecipheriv(
    ALGORITHM,
    Buffer.from(ENCRYPTION_KEY, 'hex'),
    Buffer.from(iv, 'hex')
  );

  decipher.setAuthTag(Buffer.from(authTag, 'hex'));

  let decrypted = decipher.update(encrypted, 'hex', 'utf8');
  decrypted += decipher.final('utf8');

  return JSON.parse(decrypted);
}
```

9.4 Privacy Controls

User Consent

```
CREATE TABLE user_consent (
  user_email TEXT PRIMARY KEY,
  consent_type TEXT NOT NULL, -- 'data_storage', 'ai_processing', 'public_sharing'
  granted BOOLEAN DEFAULT FALSE,
  granted_date TIMESTAMPTZ,
  revoked_date TIMESTAMPTZ,
  ip_address TEXT,
  user_agent TEXT
);
```

Data Retention

```
-- Auto-delete rejected inbox items after 90 days
CREATE FUNCTION delete_old_rejected_submissions()
RETURNS void AS $$
BEGIN
  DELETE FROM inbox
  WHERE status = 'Rejected'
  AND review_date < NOW() - INTERVAL '90 days';
END;
$$ LANGUAGE plpgsql;

-- Scheduled job (run daily)
SELECT cron.schedule('delete-old-submissions', '0 2 * * *', $$
```

```
  SELECT delete_old_rejected_submissions();
$$);
```

Right to be Forgotten (GDPR)

```
// API endpoint: DELETE /api/gdpr/delete-my-data
async function deleteUserData(userEmail: string) {
  // 1. Delete user account
  await supabase.auth.admin.deleteUser(userEmail);

  // 2. Anonymize contributions
  await supabase.from('family_members')
    .update({ added_by: 'DELETED_USER' })
    .eq('added_by', userEmail);

  await supabase.from('media')
    .update({ uploaded_by: 'DELETED_USER' })
    .eq('uploaded_by', userEmail);

  await supabase.from('stories')
    .update({ added_by: 'DELETED_USER' })
    .eq('added_by', userEmail);

  // 3. Delete personal submissions (if not approved)
  await supabase.from('inbox')
    .delete()
    .eq('submitter_email', userEmail)
    .eq('status', 'Pending');

  // 4. Log deletion
  await supabase.from('gdpr_deletions').insert({
    email: userEmail,
    deleted_at: new Date(),
    reason: 'User request'
  });
}
```

9.5 Security Best Practices Implemented

Input Validation

```
import { z } from 'zod';

// Example: Validate member creation
const MemberSchema = z.object({
  full_name: z.string().min(2).max(100),
  nickname: z.string().max(50).optional(),
  gender: z.enum(['Male', 'Female', 'Other']).optional(),
  date_of_birth: z.coerce.date().max(new Date()).optional(),
  date_of_death: z.coerce.date().max(new Date()).optional(),
  lineage: z.enum(['Father', 'Mother', 'Both']),
  status: z.enum(['Living', 'Deceased']).default('Living'),
  parent1_id: z.string().uuid().optional(),
  parent2_id: z.string().uuid().optional()
}).refine(data => {
  // Validate DOD > DOB
  if (data.date_of_birth && data.date_of_death) {
    return data.date_of_death > data.date_of_birth;
  }
  return true;
}, { message: "Date of death must be after date of birth" });

// Usage
export async function POST(req: Request) {
  const body = await req.json();

  try {
    const validatedData = MemberSchema.parse(body);
    // Proceed with valid data
  } catch (error) {
    if (error instanceof z.ZodError) {
      return Response.json({
        success: false,
        error: {
          code: 'VALIDATION_ERROR',
          message: error.errors[0].message,
          field: error.errors[0].path[0]
        }
      }, { status: 400 });
    }
  }
}
```

SQL Injection Prevention

- ✅ Use Supabase client (parameterized queries)
- ✅ Never concatenate user input into SQL
- ✅ Input validation before database operations

XSS Prevention

- ✅ React auto-escapes output

- ✅ Sanitize rich text input (DOMPurify)
- ✅ Content Security Policy headers

```
// Sanitize user input for rich text
import DOMPurify from 'dompurify';

function sanitizeStoryText(html: string): string {
  return DOMPurify.sanitize(html, {
    ALLOWED_TAGS: ['p', 'br', 'strong', 'em', 'u'],
    ALLOWED_ATTR: []
  });
}
```

CSRF Protection

- ✅ SameSite cookies
- ✅ CSRF tokens for state-changing operations
- ✅ Verify origin header

```
// CSRF token generation
import { randomBytes } from 'crypto';

function generateCSRFToken(): string {
  return randomBytes(32).toString('hex');
}

// Middleware to verify CSRF token
function verifyCSRFToken(req: Request): boolean {
  const token = req.headers.get('X-CSRF-Token');
  const cookieToken = req.cookies.get('csrf_token')?.value;

  return token === cookieToken;
}
```

File Upload Security

```
// Validate file uploads
const ALLOWED_IMAGE_TYPES = ['image/jpeg', 'image/png', 'image/webp', 'image/gif'];
const ALLOWED_VIDEO_TYPES = ['video/mp4', 'video/quicktime'];
const MAX_FILE_SIZE = 100 * 1024 * 1024; // 100MB

function validateUpload(file: File): { valid: boolean; error?: string } {
  // Check file type
  if (![...ALLOWED_IMAGE_TYPES, ...ALLOWED_VIDEO_TYPES].includes(file.type)) {
    return { valid: false, error: 'Unsupported file type' };
  }

  // Check file size
  if (file.size > MAX_FILE_SIZE) {
    return { valid: false, error: 'File too large' };
  }

  // Verify file signature (magic bytes)
  const reader = new FileReader();
  reader.onload = (e) => {
    const arr = new Uint8Array(e.target.result as ArrayBuffer).subarray(0, 4);
    const header = Array.from(arr).map(b => b.toString(16)).join('');

    // JPEG: ffd8ff
    // PNG: 89504e47
    // WebP: 52494646

    if (!isValidFileSignature(header, file.type)) {
      return { valid: false, error: 'File signature mismatch' };
    }
  };
  reader.readAsArrayBuffer(file.slice(0, 4));

  return { valid: true };
}

function isValidFileSignature(header: string, mimeType: string): boolean {
  const signatures: Record<string, string[]> = {
    'image/jpeg': ['ffd8ff'],
    'image/png': ['89504e47'],
    'image/webp': ['52494646'],
    'video/mp4': ['00000018', '00000020']
  };

  const validSignatures = signatures[mimeType] || [];
  return validSignatures.some(sig => header.startsWith(sig));
}
```

API Security Headers

```
// next.config.js
module.exports = {
  async headers() {
    return [
      {
        source: '/:path*',
        headers: [
```

```
          {
            key: 'X-DNS-Prefetch-Control',
            value: 'on'
          },
          {
            key: 'Strict-Transport-Security',
            value: 'max-age=63072000; includeSubDomains; preload'
          },
          {
            key: 'X-Frame-Options',
            value: 'SAMEORIGIN'
          },
          {
            key: 'X-Content-Type-Options',
            value: 'nosniff'
          },
          {
            key: 'X-XSS-Protection',
            value: '1; mode=block'
          },
          {
            key: 'Referrer-Policy',
            value: 'strict-origin-when-cross-origin'
          },
          {
            key: 'Permissions-Policy',
            value: 'camera=(), microphone=(), geolocation=()'
          },
          {
            key: 'Content-Security-Policy',
            value: [
              "default-src 'self'",
              "script-src 'self' 'unsafe-eval' 'unsafe-inline'",
              "style-src 'self' 'unsafe-inline'",
              "img-src 'self' data: https:",
              "font-src 'self' data:",
              "connect-src 'self' https://api.supabase.co https://*.r2.dev https://api.cloudflare.com"
            ].join('; ')
          }
        ]
      }
    ];
  }
};
```

9.6 Password Security

Password Requirements

```
const PASSWORD_REQUIREMENTS = {
  minLength: 8,
  maxLength: 128,
  requireUppercase: true,
  requireLowercase: true,
  requireNumber: true,
  requireSpecial: true,
  preventCommon: true
};

function validatePassword(password: string): { valid: boolean; errors: string[] } {
  const errors: string[] = [];

  if (password.length < PASSWORD_REQUIREMENTS.minLength) {
    errors.push(`Password must be at least ${PASSWORD_REQUIREMENTS.minLength} characters`);
  }

  if (PASSWORD_REQUIREMENTS.requireUppercase && !/[A-Z]/.test(password)) {
    errors.push('Password must contain at least one uppercase letter');
  }

  if (PASSWORD_REQUIREMENTS.requireLowercase && !/[a-z]/.test(password)) {
    errors.push('Password must contain at least one lowercase letter');
  }

  if (PASSWORD_REQUIREMENTS.requireNumber && !/[0-9]/.test(password)) {
    errors.push('Password must contain at least one number');
  }

  if (PASSWORD_REQUIREMENTS.requireSpecial && !/[!@#$%^&*(),.?":{}|<>]/.test(password)) {
    errors.push('Password must contain at least one special character');
  }

  // Check against common passwords
  const commonPasswords = ['password', '123456', 'qwerty', 'admin'];
  if (commonPasswords.includes(password.toLowerCase())) {
    errors.push('Password is too common');
  }

  return {
    valid: errors.length === 0,
    errors
  };
}
```

Password Hashing (Supabase handles this)

Supabase uses bcrypt with 10 rounds by default. No custom implementation needed.

9.7 Session Management

```
// Session configuration
const SESSION_CONFIG = {
  maxAge: 30 * 24 * 60 * 60, // 30 days
  secure: process.env.NODE_ENV === 'production',
  httpOnly: true,
  sameSite: 'lax' as const,
  path: '/'
};

// Set session cookie
function setSessionCookie(res: Response, token: string) {
  res.headers.set('Set-Cookie',
    `session=${token}; Max-Age=${SESSION_CONFIG.maxAge}; Path=${SESSION_CONFIG.path}; ${SESSION_CONFIG.secure ? 'Secure;' : ''} HttpOnly; SameSite=${SESSION_CONFIG.sameSite}`
  );
}

// Clear session cookie (logout)
function clearSessionCookie(res: Response) {
  res.headers.set('Set-Cookie',
    `session=; Max-Age=0; Path=${SESSION_CONFIG.path}`
  );
}
```

9.8 Audit Logging

```
// Log security-sensitive actions
async function logSecurityEvent(event: {
  action: string;
  user_email: string;
  resource_type: string;
  resource_id?: string;
  ip_address: string;
  user_agent: string;
  success: boolean;
  error_message?: string;
}) {
  await supabase.from('security_log').insert({
    ...event,
    timestamp: new Date()
  });
}

// Usage examples
await logSecurityEvent({
  action: 'LOGIN',
  user_email: 'admin@example.com',
  resource_type: 'auth',
  ip_address: req.ip,
  user_agent: req.headers.get('user-agent'),
  success: true
});

await logSecurityEvent({
  action: 'DELETE_MEMBER',
  user_email: 'admin@example.com',
  resource_type: 'family_member',
  resource_id: 'uuid',
  ip_address: req.ip,
  user_agent: req.headers.get('user-agent'),
  success: true
});
```

9.9 Threat Model

Identified Threats

Threat 1: Unauthorized Access

- **Risk**: Malicious actor gains access to family data
- **Mitigation**:
  - Strong password requirements
  - 2FA for admins
  - Rate limiting on login attempts
  - Session timeout after 30 days
  - IP-based access monitoring

Threat 2: Data Breach

- **Risk**: Database compromised, sensitive data exposed
- **Mitigation**:
  - Encryption at rest (AES-256)
  - Encryption in transit (TLS 1.3)
  - Sensitive fields double-encrypted
  - Regular security audits
  - Minimal data retention

Threat 3: Malicious File Upload

- **Risk**: Malware uploaded via media upload
- **Mitigation**:
  - File type validation
  - File signature verification
  - Size limits enforced
  - Files stored separately from app
  - No executable files allowed
  - Virus scanning (future)

Threat 4: SQL Injection

- **Risk**: Database manipulated via crafted input
- **Mitigation**:
  - Parameterized queries only
  - ORM/query builder (Supabase)
  - Input validation
  - Principle of least privilege

Threat 5: XSS Attack

- **Risk**: Malicious script injected into pages
- **Mitigation**:
  - React auto-escaping
  - Content Security Policy
  - DOMPurify for rich text
  - No inline scripts

Threat 6: CSRF Attack

- **Risk**: Unauthorized actions performed
- **Mitigation**:
  - CSRF tokens
  - SameSite cookies
  - Origin verification
  - State-changing operations require POST

Threat 7: Account Takeover

- **Risk**: User account compromised
- **Mitigation**:
  - 2FA available
  - Email verification required
  - Password reset via email only
  - Unusual activity alerts

Threat 8: Insider Threat

- **Risk**: Admin abuses privileges
- **Mitigation**:
  - All actions logged
  - Multiple admins (separation of duties)
  - Change log tracks modifications
  - Super Admin required for deletions

Threat 9: DDoS Attack

- **Risk**: Service unavailable
- **Mitigation**:
  - Cloudflare DDoS protection
  - Rate limiting per user
  - CDN caching
  - Graceful degradation

Threat 10: Data Loss

- **Risk**: Accidental deletion or corruption
- **Mitigation**:
  - Soft deletes (no hard deletes)
  - Automated daily backups
  - Version history
  - 3-2-1 backup strategy
  - Point-in-time recovery

9.10 Security Checklist

Development Phase

- ☐ All secrets in environment variables
- ☐ No hardcoded credentials
- ☐ Input validation on all endpoints
- ☐ Output encoding/escaping
- ☐ Parameterized database queries
- ☐ File upload validation
- ☐ Rate limiting implemented

- [ ] CSRF protection enabled
- [ ] Security headers configured

Deployment Phase

- [ ] HTTPS enforced
- [ ] Security headers active
- [ ] Database backups automated
- [ ] Secrets rotated
- [ ] Monitoring enabled
- [ ] Logging configured
- [ ] Error handling (no info leakage)
- [ ] Dependencies updated

Ongoing Operations

- [ ] Regular security audits
- [ ] Dependency updates (monthly)
- [ ] Log review (weekly)
- [ ] Backup verification (monthly)
- [ ] Penetration testing (annually)
- [ ] Incident response plan updated
- [ ] User permissions reviewed
- [ ] Compliance check (GDPR)

16. IMPLEMENTATION ROADMAP

16.1 12-Week Implementation Plan

PHASE 0: Setup (Week 1 - After Exam)

Day 1: Create Accounts (30 minutes)

```
Tasks:
✓ Sign up for Supabase (supabase.com)
✓ Create Cloudflare account (cloudflare.com)
✓ Create GitHub account/repo (github.com)
✓ Sign up for Vercel (vercel.com)
✓ Create Tally account (tally.so)
✓ Sign up for Resend (resend.com)

Deliverables:
- All accounts created
- Email addresses verified
- Account credentials saved in password manager
```

Day 2: Database Setup (1 hour)

```
Tasks:
✓ Create new Supabase project
✓ Copy SQL schema from SRS
✓ Run schema in Supabase SQL editor
✓ Enable Row Level Security (RLS)
✓ Create admin_users table
✓ Insert your email as Super Admin
✓ Generate API keys (anon + service role)
✓ Save connection strings

Deliverables:
- Database schema deployed
- API keys saved
- Test connection successful
```

Day 3: Storage Setup (1 hour)

```
Tasks:
✓ Create Cloudflare R2 bucket: "roots-of-dewangan"
✓ Generate R2 access credentials
✓ Configure CORS policy
✓ Test upload via API
✓ Enable public access for media
✓ Create folder structure: photos/, videos/, audio/, docs/, thumbs/

Deliverables:
- R2 bucket configured
- Test file uploaded successfully
- Public URL working
```

Day 4: Forms Setup (1 hour)

```
Tasks:
✓ Create 5 Tally forms:
  1. Add Family Member
  2. Share a Story
```

```
   3. Upload Photo/Video
   4. Document Event
   5. Record Tradition
✓ Configure webhooks to point to /api/webhook/submission
✓ Customize form branding (colors, logo)
✓ Test form submission
✓ Verify webhook receives data

Deliverables:
- 5 public forms live
- Webhooks configured
- Test submissions working
```

Day 5: Deploy Web App (2 hours)

```
Tasks:
✓ Clone Next.js starter template
✓ Install dependencies (npm install)
✓ Configure environment variables (.env.local)
✓ Connect to Supabase
✓ Create basic homepage
✓ Deploy to Vercel
✓ Configure custom domain (optional)

Deliverables:
- Working homepage at https://[your-app].vercel.app
- Database connection verified
- Environment variables set
```

Week 1 Deliverable: Working Prototype

- Can add 1 member via form
- Data visible in Supabase dashboard
- Homepage loads successfully

PHASE 1: Core Features (Weeks 2-4)

Week 2: Family Tree Visualization

Goals:

- Interactive family tree component
- Member CRUD operations
- Relationship linking

Tasks:

```
Day 1-2: Setup React Flow
- Install react-flow-renderer
- Create basic tree layout
- Fetch members from database
- Display nodes with photos

Day 3-4: Interactivity
- Click node to view profile
- Zoom/pan controls
- Filter by lineage/generation
- Search functionality

Day 5: Member Management
- Create member form
- Edit member form
- Delete member (soft delete)
- Validation

Deliverables:
- Interactive family tree
- Add/Edit/Delete members
- 10 test members added
```

Week 3: Media Gallery

Goals:

- Upload photos/videos
- Automatic compression
- Gallery view with filters

Tasks:

```
Day 1-2: Upload System
- File upload component
- Integration with R2
- Progress indicators
- Error handling

Day 3: Compression
- Integrate Sharp library
- Convert to WebP
- Generate thumbnails
- Log compression stats
```

```
Day 4-5: Gallery UI
- Grid view of photos
- Lightbox for full-size
- Filter by date/person/tags
- Tag people in photos

Deliverables:
- Working upload system
- 50 photos uploaded
- Compression working (70% reduction)
- Gallery with filters
```

Week 4: Stories & Timeline

Goals:

- Story creation interface
- Event timeline
- Link stories to members

Tasks:

```
Day 1-2: Story Form
- Rich text editor
- Audio upload option
- Tag members
- Categorize by type

Day 3: Event Management
- Create event form
- Link attendees
- Add photos to events
- Event detail page

Day 4-5: Timeline View
- Chronological display
- Filter by person/type
- Visual timeline component
- Decade/year/month zoom

Deliverables:
- 10 stories created
- 5 events documented
- Timeline view working
```

Phase 1 Milestone: Core Features Complete

- 50 members documented
- 100 photos uploaded
- 10 stories preserved
- Interactive tree working

PHASE 2: AI Integration (Weeks 5-6)

Week 5: Image Processing

Goals:

- Automatic image compression
- AI captioning
- Object detection

Tasks:

```
Day 1-2: Compression Pipeline
- Set up Cloudflare Worker
- Implement WebP conversion
- Generate thumbnails automatically
- Update database with stats

Day 3-4: AI Tagging
- Integrate Cloudflare AI
- Implement image captioning
- Object detection
- Store results in database

Day 5: Face Detection
- Detect faces in photos
- Extract coordinates
- Store face data
- Basic matching (admin confirms)

Deliverables:
- All uploads automatically compressed
- AI captions for 100 photos
- Face detection working
```

Week 6: Story Analysis

Goals:

- AI theme extraction

- Auto-summarization
- Semantic search

Tasks:

```
Day 1-2: Theme Extraction
- Integrate LLaMA 2
- Extract themes from stories
- Sentiment analysis
- Store AI results

Day 3-4: Summarization
- Generate 2-sentence summaries
- Update existing stories
- Display in story cards

Day 5: Semantic Search
- Generate embeddings
- Store in vector column
- Implement similarity search
- Search UI

Deliverables:
- All stories have AI themes
- Summaries generated
- Semantic search working
```

Phase 2 Milestone: AI Features Live

- AI processes all new uploads
- Themes auto-extracted
- Semantic search functional

PHASE 3: Family Engagement (Weeks 7-8)

Week 7: Submission System

Goals:

- Public forms integrated
- Inbox review dashboard
- Email notifications

Tasks:

```
Day 1-2: Webhook Handler
- Create /api/webhook/submission
- Parse Tally form data
- Insert into inbox table
- Handle file uploads

Day 3-4: Inbox Dashboard
- List pending submissions
- View submission details
- Approve/reject buttons
- Link to existing records

Day 5: Notifications
- Email on new submission
- Email on approval/rejection
- Weekly digest
- Birthday reminders

Deliverables:
- 5 forms connected
- Inbox dashboard working
- Email notifications active
```

Week 8: Mobile PWA

Goals:

- Progressive Web App
- Camera integration
- Offline support
- Push notifications

Tasks:

```
Day 1-2: PWA Setup
- Create manifest.json
- Service worker
- Install prompt
- Offline fallback

Day 3-4: Camera Integration
- Camera access
- Take photo in-app
- Immediate upload
- Preview before submit

Day 5: Offline Features
```

```
- Cache static assets
- Queue offline submissions
- Sync when online
- Offline indicator

Deliverables:
- Installable PWA
- Camera upload working
- Offline mode functional
```

Phase 3 Milestone: Engagement Ready

- Family can submit via forms
- Mobile upload working
- Admin reviews easily

PHASE 4: Advanced Features (Weeks 9-12)

Week 9: Analytics Dashboard

Goals:

- Family statistics
- Occupation trends
- Migration map
- Generation comparison

Tasks:

```
Day 1-2: Statistics Page
- Total counts
- Living vs deceased
- Gender distribution
- Age statistics

Day 3: Occupation Trends
- Chart by generation
- Category breakdown
- Timeline visualization

Day 4: Migration Map
- Integrate Leaflet
- Plot locations
- Migration routes
- Interactive markers

Day 5: Generation Comparison
- Side-by-side stats
- Education evolution
- Occupation shifts

Deliverables:
- Analytics dashboard live
- All charts working
- Migration map interactive
```

Week 10: Audio & Documents

Goals:

- Audio story upload
- AI transcription
- Document vault with OCR

Tasks:

```
Day 1-2: Audio Upload
- Audio file support
- Player component
- Waveform visualization

Day 3: Transcription
- Integrate Whisper
- Auto-transcribe uploads
- Edit transcript UI

Day 4-5: Document Vault
- PDF upload
- DOCX support
- OCR text extraction
- Searchable documents

Deliverables:
- 5 audio stories transcribed
- Document vault working
- OCR functional
```

Week 11: Traditions & Reminders

Goals:

- Tradition documentation

- Recipe book
- Reminder system

Tasks:

```
Day 1-2: Traditions Module
- Tradition form
- Link practitioners
- Categorize by type
- Timeline of practice

Day 3: Recipe Book
- Recipe format
- Ingredient lists
- Instructions
- Photo integration

Day 4-5: Reminders
- Birthday reminders
- Anniversary alerts
- Festival calendar
- Email notifications

Deliverables:
- 5 traditions documented
- 3 recipes added
- Reminders working
```

Week 12: Backup & Export

Goals:

- Automated backups
- GEDCOM export
- PDF reports
- Data completeness tracker

Tasks:

```
Day 1-2: Backup System
- GitHub Actions workflow
- Daily database export
- Store in private repo
- Email confirmation

Day 3: GEDCOM Export
- Generate .ged file
- Include all members
- Relationships mapped
- Download link

Day 4: PDF Reports
- Template design
- Generate with Puppeteer
- Include photos/stories
- Customizable options

Day 5: Data Quality
- Completeness tracker
- Missing data report
- Suggestions UI
- Admin dashboard widget

Deliverables:
- Daily backups running
- GEDCOM export working
- PDF generation functional
- Data quality dashboard
```

Phase 4 Milestone: Production Ready

- All core features complete
- Automated backups running
- Export functions working
- System stable

16.2 Detailed Week-by-Week Checklist

Week 1 Checklist

Setup Tasks:

- [ ] Create Supabase account
- [ ] Create Cloudflare account (R2 + AI)
- [ ] Create GitHub repository
- [ ] Create Vercel account
- [ ] Create Tally account
- [ ] Create Resend account
- [ ] Run database schema SQL
- [ ] Enable RLS policies

- [ ] Insert admin user
- [ ] Create R2 bucket
- [ ] Configure CORS
- [ ] Create 5 Tally forms
- [ ] Clone Next.js template
- [ ] Configure environment variables
- [ ] Deploy to Vercel
- [ ] Test database connection
- [ ] Test file upload to R2
- [ ] Test form submission
- [ ] Verify webhook receives data

Success Criteria:

- [x] All services accessible
- [x] Database schema deployed
- [x] Can add member via form
- [x] Data appears in database
- [x] App deployed and live

Week 2 Checklist

Family Tree:

- [ ] Install React Flow
- [ ] Create family tree component
- [ ] Fetch members from Supabase
- [ ] Display nodes with names/photos
- [ ] Implement zoom/pan controls
- [ ] Add click to view profile
- [ ] Implement filters (lineage, generation, status)
- [ ] Add search bar
- [ ] Create member profile page
- [ ] Build add member form
- [ ] Build edit member form
- [ ] Implement soft delete
- [ ] Add form validation
- [ ] Link parent-child relationships
- [ ] Auto-calculate generation

Success Criteria:

- [x] Tree displays 10+ members
- [x] Can navigate by clicking nodes
- [x] Can add new members
- [x] Can edit existing members
- [x] Filters work correctly

Week 3 Checklist

Media Gallery:

- [ ] Create upload component
- [ ] Integrate with R2 API
- [ ] Implement progress bar
- [ ] Add file type validation
- [ ] Add size validation
- [ ] Install Sharp library
- [ ] Implement WebP conversion
- [ ] Generate thumbnails (400x400)
- [ ] Log compression stats
- [ ] Create media gallery page
- [ ] Implement grid view
- [ ] Add lightbox for full-size
- [ ] Add filters (date, person, type)
- [ ] Implement tagging people
- [ ] Create media detail page
- [ ] Add download button

Success Criteria:

- [x] Can upload photos
- [x] Auto-compression works (70% reduction)
- [x] Thumbnails generated
- [x] Gallery displays photos
- [x] Can tag people in photos

Week 4 Checklist

Stories & Timeline:

- [ ] Create story form
- [ ] Add rich text editor (optional)
- [ ] Implement audio upload
- [ ] Add member tagging
- [ ] Add theme selection
- [ ] Create story list page
- [ ] Implement filters (type, person, theme)
- [ ] Create story detail page
- [ ] Add comment system (optional)
- [ ] Create event form
- [ ] Link attendees to events
- [ ] Create event detail page
- [ ] Build timeline component
- [ ] Implement chronological sorting
- [ ] Add zoom controls (decade/year/month)
- [ ] Add visual markers

Success Criteria:

- ✅ Can create stories
- ✅ Can document events
- ✅ Timeline displays chronologically
- ✅ Can filter timeline
- ✅ 10 stories documented

16.3 Resource Allocation

Time Investment

Setup Phase (Week 1):

- Total: 5 hours
- Daily: 30 min - 2 hours

Development Phase (Weeks 2-12):

- Total: 80-110 hours
- Weekly: 8-12 hours
- Daily: 1-2 hours

Maintenance (Ongoing):

- Weekly: 1-2 hours
- Monthly: 4-8 hours

Budget Breakdown

Year 1:

```
Supabase: $0 (Free tier)
Cloudflare R2: $0 (Free 10GB)
Cloudflare AI: $0 (Free 10K/day)
Vercel: $0 (Hobby tier)
Tally: $0 (Free unlimited)
Resend: $0 (Free 3K emails/mo)
Domain (optional): $12/year

TOTAL: $0-12/year
```

Year 2-5:

```
Potential costs only if exceeding free tiers:
Supabase Pro: $25/month (if >500MB DB)
R2 Storage: $0.015/GB beyond 10GB
AI requests: Still free (10K/day sufficient)

Estimated: $0-300/year
```

Skills Required

Essential:

- Basic JavaScript/TypeScript
- React basics
- SQL basics (copy-paste queries)
- Git basics (commit, push)

Helpful but not required:

- Next.js framework
- API integration
- Database design

- UI/UX design

Not required (we provide):

- Advanced backend
- DevOps
- AI/ML knowledge
- Scalability expertise

16.4 Risk Mitigation

Technical Risks

Risk 1: Free tier exceeded

- **Probability:** Low (designed for free tier)
- **Impact:** Medium
- **Mitigation:**
  - Monitor usage monthly
  - Optimize queries
  - Compress media aggressively
  - Archive old data
- **Contingency:** Upgrade to paid ($25/month)

Risk 2: Service shutdown

- **Probability:** Very low (Supabase, Cloudflare stable)
- **Impact:** High
- **Mitigation:**
  - Daily backups to GitHub
  - Data exportable (CSV, JSON)
  - Open standards (PostgreSQL, S3)
- **Contingency:** Migrate to alternative (documented)

Risk 3: Data loss

- **Probability:** Very low
- **Impact:** Critical
- **Mitigation:**
  - Automated daily backups
  - Version history (Supabase)
  - 3-2-1 backup strategy
  - Soft deletes only
- **Contingency:** Restore from backup

Risk 4: Performance degradation

- **Probability:** Medium (after 5+ years)
- **Impact:** Medium
- **Mitigation:**
  - Efficient indexes
  - Pagination
  - Lazy loading
  - Archive old data
- **Contingency:** Upgrade tier or optimize

Organizational Risks

Risk 5: Low family engagement

- **Probability:** Medium
- **Impact:** High
- **Mitigation:**
  - Easy submission (forms)
  - Mobile-first design
  - Gamification
  - Weekly prompts
- **Contingency:** Admin enters data manually

Risk 6: Admin unavailability

- **Probability:** Medium
- **Impact:** High
- **Mitigation:**
  - Train secondary admin
  - Comprehensive documentation
  - Automated processes
  - Simple UI
- **Contingency:** Temporary admin access

Risk 7: Disputes over facts

- **Probability:** High
- **Impact:** Medium
- **Mitigation:**
  - Dispute resolution process

    - Source citation required
    - Change log tracks all edits
    - Multiple verification
- **Contingency:** Mark as "disputed" with both versions

16.5 Success Metrics

Technical Metrics

Month 1:

- [ ] System uptime: >99%
- [ ] Page load time: <2 seconds
- [ ] Zero data loss
- [ ] All forms functional

Month 3:

- [ ] 50+ members documented
- [ ] 100+ photos uploaded
- [ ] 20+ stories preserved
- [ ] Compression ratio: >60%

Month 6:

- [ ] 100+ members
- [ ] 300+ photos
- [ ] 50+ stories
- [ ] 10+ active contributors

Year 1:

- [ ] 150+ members
- [ ] 500+ photos
- [ ] 75+ stories
- [ ] 80% data completeness

Engagement Metrics

Month 1:

- [ ] 3+ family members registered
- [ ] 5+ submissions via forms
- [ ] 1+ stories shared

Month 3:

- [ ] 10+ family members registered
- [ ] 20+ submissions via forms
- [ ] 10+ stories shared
- [ ] 50+ photos uploaded

Month 6:

- [ ] 15+ active contributors
- [ ] 5+ submissions per week
- [ ] 1+ story per week

Year 1:

- [ ] 20+ contributors
- [ ] Weekly activity
- [ ] Family reunion documented
- [ ] 1 AI-generated book draft

16.6 Post-Launch Activities

Month 1-3: Stabilization

- Monitor error logs daily
- Fix bugs as reported
- Optimize slow queries
- Gather user feedback
- Refine UI based on usage

Month 4-6: Enhancement

- Add requested features
- Improve mobile experience
- Expand AI capabilities
- Create video tutorials
- Launch engagement campaigns

Month 7-12: Expansion

- Implement priority features (DNA integration, WhatsApp bot)
- Generate first family book
- Organize data collection drives
- Add more traditions/recipes
- Plan year 2 roadmap

Year 2+: Maintenance & Growth

- Continue engagement
- Add new features quarterly
- Keep dependencies updated
- Scale infrastructure as needed
- Preserve for next generation

16.7 Training & Documentation

Admin Training Materials

Video Tutorials (to create):

1. System Overview (10 min)
2. Adding Family Members (5 min)
3. Uploading Photos (5 min)
4. Reviewing Submissions (8 min)
5. Resolving Disputes (6 min)
6. Exporting Data (4 min)
7. Managing Backups (5 min)

Written Guides:

- Quick Start Guide
- Admin Manual
- Contributor Guide
- Troubleshooting FAQ
- Backup & Recovery Guide

User Documentation

For Family Members:

- How to submit a member (form guide)
- How to upload photos (step-by-step)
- How to share stories (examples)
- How to install PWA on phone
- Privacy & data usage

16.8 Next Steps After This Document

Immediate (Today):

1. Save this SRS document
2. Review entire document
3. Bookmark important sections
4. List any questions

After Exam (Week 1):

1. Follow Day 1-5 setup guide
2. Create all accounts
3. Run database schema
4. Deploy basic app
5. Test with 1 member

Weeks 2-12:

1. Follow weekly checklists
2. Track progress in spreadsheet
3. Ask questions as needed
4. Test each feature
5. Celebrate milestones

Year 1+:

1. Engage family members
2. Document heritage
3. Preserve memories
4. Build legacy
5. Pass to next generation

END OF IMPLEMENTATION ROADMAP

This completes the Implementation section. The SRS document is now complete with all major sections covered.